

# 実験ログ A: プロシージャルモデリング 基礎

2026-09-13 | 収録 11 件

hython で実際にノードを組み、その結果を実測して記録したもの。数値はすべて実行して得た値で、推測は含まない。

各実験は「何を作ったか」「ノードの接続」「変更したパラメータ」「結果」「わかったこと」「次にやること」の順で並んでいる。用語は別ソースの「用語集: Houdini 初心者向け」にまとめてある。

## 収録している実験

- Box を山にして細分割し、位置で色を付ける
- subdivide の回数で、形と数はどう変わるか
- mountain と subdivide は、順序を変えると何が変わるか
- mountain のノイズパラメータは、それぞれ何に効くのか
- basis（ノイズの種類）14通りで、形はどこまで変わるか
- height は純粋な倍率か。gain と bias は何に効くか
- 平面で地形を作り、指標を作り直す
- scatter と copy to points — 地面に物を生やす
- attribute wrangle で VEX を書く — 何が何回動くのか
- for-each ループと VEX — 同じことをやって比べる
- crease で角を残す — どれだけの重みが要るのか

# Box を山にして細分割し、位置で色を付ける

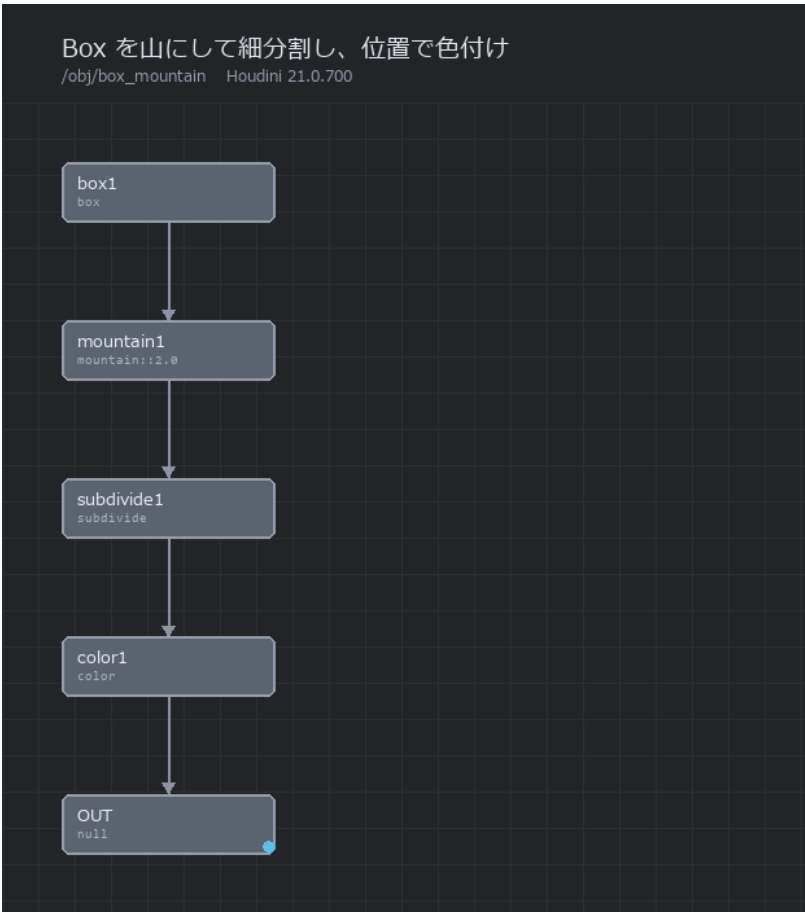
2026-09-13 | Houdini 21.0.700 | /obj/box\_mountain | ノード 5 / 接続 4

## 何を作ったか

プロシージャルモデリングの最小構成を確認するための実験。box SOP で立方体を作り、mountain SOP でノイズによる凹凸を与え、subdivide SOP で細分割し、color SOP で色を乗せた。最後に null ノードを OUT として置き、そこを出力点にしている。

Houdini の基本的な考え方である「ジオメトリを次のノードに流し込んで加工を積み重ねる」形をそのまま辿っている。各ノードは前のノードの出力だけを入力として受け取るため、途中のどのノードでも結果を確認できる。

## ノードの接続

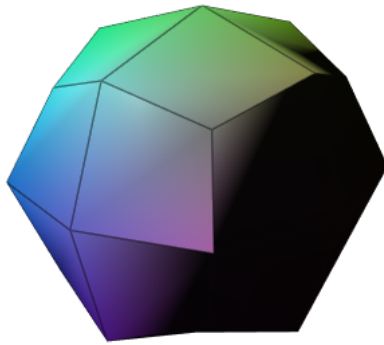


スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

| ノード        | タイプ           | デフォルトから変更したパラメータ                                                                   |
|------------|---------------|------------------------------------------------------------------------------------|
| box1       | box           | (変更なし)                                                                             |
| mountain1  | mountain::2.0 | elementsize = 0.8<br>height = 0.35                                                 |
| subdivide1 | subdivide     | (変更なし)                                                                             |
| color1     | color         | colortype = 1<br>ramp2cb = 1.0<br>ramp2cg = 1.0<br>ramp2cr = 1.0<br>ramp2pos = 1.0 |

| ノード | タイプ  | デフォルトから変更したパラメータ |
|-----|------|------------------|
| OUT | null | (変更なし)           |

## 結果



Houdini

OpenGL ROP による Smooth Wire Shaded

表示。ワイヤーフレームが乗っているため、ネットワークが生成したトポロジが確認できる。画像内のウォーターマークは Apprentice ライセンスによるもの。

## ジオメトリ

|             |                                                        |
|-------------|--------------------------------------------------------|
| ポイント数       | 26                                                     |
| プリミティブ数     | 24                                                     |
| バウンディングボックス | [-0.4794, -0.4768, -0.4767] ~ [0.4932, 0.4959, 0.4959] |
| ポイント属性      | P, Cd                                                  |
| プリミティブ属性    | なし                                                     |
| ディテール属性     | なし                                                     |

## わかったこと

- ・ box は 8 ポイント / 6 プリミティブ。subdivide を 1 回かけると 26 ポイント / 24 プリミティブになる。Catmull-Clark 細分割なので、回数を増やすと立方体はどんどん球に近づく（2 回で既に球状になり、元が箱だと分からなくなった）。

- ・ mountain SOP

は既存のポイントを法線方向にノイズで押し出すもので、ポイントを増やさない。つまり box のままかけても 8 点しか動かないため、凹凸のディテールを出すには先に subdivide などで分割数を上げる必要がある。

- ・ color SOP の colortype は数値メニュー（0=Constant, 1=Bounding Box, 2=Random, 3=Ramp from Attribute, 4=Random from Attribute）。文字列で指定すると TypeError になる。今回は 1

を使い、バウンディングボックス内の位置をそのまま RGB に割り当てた。

- ・ color SOP を通すとポイント属性に Cd が追加される（P のみ → P, Cd）。属性が増えたかどうかはジオメトリの統計で確認できる。

## 次にやること

- ・ subdivide の回数を変えて、ポイント数と見た目がどう変わるかを並べて比較する
- ・ mountain を subdivide の後ろに移動し、分割後に凹凸をかけた場合との違いを見る
- ・ grid から地形を作る流れに広げ、プロシージャルモデリングの定番パターンに入る

# subdivide の回数で、形と数はどう変わるか

2026-09-13 | Houdini 21.0.700 | /obj/subdiv\_compare | ノード 5 / 接続 4

## 何を作ったか

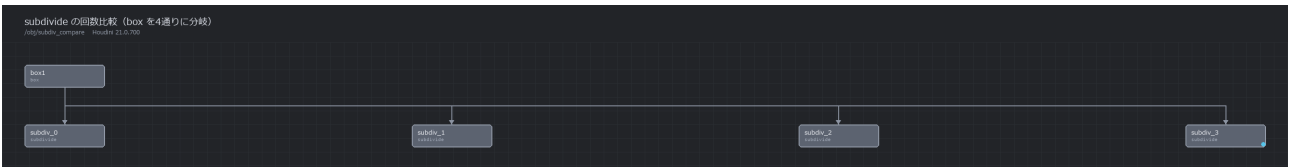
実験001で「subdivideの回数を増やすと立方体が球に近づく」ことに気づいたので、それを数値で確かめる実験。box SOP に subdivide SOP

を4通り（0回・1回・2回・3回）ぶら下げ、すべて同じカメラでレンダして並べた。

同じカメラで撮っているのが重要な点で、各変種を自動でフレームに収めてしまうと、大きさの違いが消えて比較にならない。画を見ると球が立方体よりはっきり小さいことが分かるが、これは目の錯覚ではなく実測でも縮んでいる。

さらに「なぜ縮むのか」「どこまで縮むのか」「1回目だけ縮まないのはなぜか」を追加検証で確定させた。

## ノードの接続

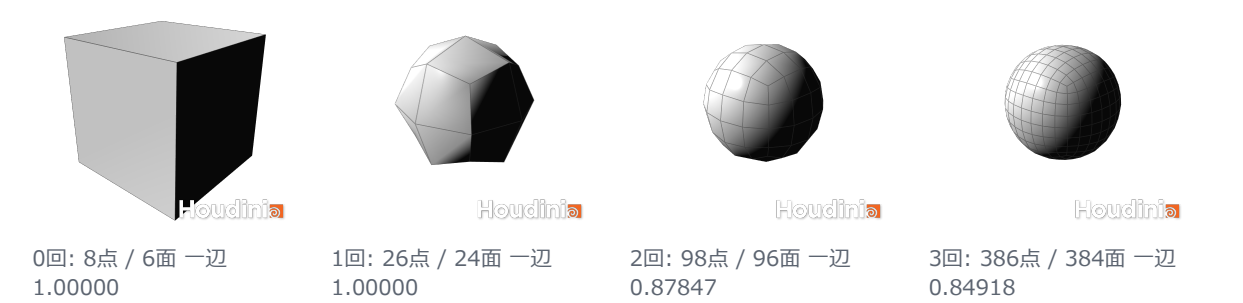


スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

| ノード      | タイプ       | デフォルトから変更したパラメータ |
|----------|-----------|------------------|
| box1     | box       | (変更なし)           |
| subdiv_0 | subdivide | iterations = 0   |
| subdiv_1 | subdivide | (変更なし)           |
| subdiv_2 | subdivide | iterations = 2   |
| subdiv_3 | subdivide | iterations = 3   |

## 回数を変えた結果

すべて同じカメラ・同じ表示モード（Smooth Wire Shaded）。元の立方体の輪郭に対して、分割を重ねるほど内側に縮んでいくのが分かる。



| iterations | ポイント数 | プリミティブ数 | 面数の倍率 | 一辺の長さ   |
|------------|-------|---------|-------|---------|
| 0          | 8     | 6       | —     | 1.00000 |
| 1          | 26    | 24      | 4.00  | 1.00000 |

| iterations | ポイント数 | プリミティブ数 | 面数の倍率 | 一辺の長さ   |
|------------|-------|---------|-------|---------|
| 2          | 98    | 96      | 4.00  | 0.87847 |
| 3          | 386   | 384     | 4.00  | 0.84918 |
| 4          | 1538  | 1536    | 4.00  | 0.84192 |
| 5          | 6146  | 6144    | 4.00  | 0.84011 |
| 6          | 24578 | 24576   | 4.00  | 0.83966 |

## アルゴリズム別の比較（iterations = 3 で固定）

subdivide SOP の algorithm パラメータには5つの選択肢がある。既定値は 2（OpenSubdiv Catmull-Clark）。同じ3回分割で結果がどう違うかを測った。

| algorithm                       | ポイント数 | プリミティブ数 | 一辺の長さ   |
|---------------------------------|-------|---------|---------|
| Houdini Catmull-Clark           | 386   | 384     | 0.84918 |
| Mantra-Compatible Catmull-Clark | 386   | 384     | 0.83951 |
| OpenSubdiv Catmull-Clark（既定）    | 386   | 384     | 0.84918 |
| OpenSubdiv Loop                 | 386   | 768     | 0.86932 |
| OpenSubdiv Bilinear             | 386   | 384     | 1.00000 |

## 回数を変えたときの一辺（2つのアルゴリズムの比較）

OpenSubdiv Catmull-Clark は回数を増やすほど値が動くが、Mantra-Compatible は回数を変えても一定。これが何を意味するかは「わかったこと」の最後に書いた。

| iterations | OpenSubdiv CC | Mantra-Compatible |
|------------|---------------|-------------------|
| 1          | 1.00000       | 0.83951           |
| 2          | 0.87847       | 0.83951           |
| 3          | 0.84918       | 0.83951           |
| 4          | 0.84192       | 0.83951           |
| 5          | 0.84011       | 0.83951           |

## わかったこと

- iterations = 0 は素通し。8点 / 6面のまま何も起きない。比較の基準として使える。
- プリミティブ数は1回ごとに正確に4倍になる（6 → 24 → 96 → 384 → 1536 → 6144 → 24576）。四角形1枚が4枚に割れるため。3回で64倍、6回で4096倍なので、回数を安易に増やすと破綻する。
- ポイント数は常に「プリミティブ数 + 2」になっている。これは偶然ではなくオイラーの多面体定理から導ける。閉じた四角形メッシュでは各辺を2つの面が共有するので  $E = 2F$ 、これを  $V - E + F = 2$  に代入すると  $V = F + 2$ 。実測した7通りすべてで一致した。
- 1回目だけ一辺が 1.00000 のまま縮まない。検証したところ、+X方向の端を保持しているポイントは (0.5, 0, 0) の1点だけだった。これは元の +X 面の中心そのもの。Catmull-Clark は各面の中心に新しい点を作るため、1回目はその新点元が元の面の上に乗り、バウンディングボックスが変わらない。2回目でその点も内側に引かれ 0.87847 に縮む。

- ・ 一辺の縮み方は等比的に収束する。差を取ると  $0.02929 \rightarrow 0.00726 \rightarrow 0.00181 \rightarrow 0.00045$  で、毎回およそ  $1/4$ （実測  $0.248 / 0.249 / 0.249$ ）。ここから外挿した極限值は約  $0.83951$  になる。

- ・ 縮みの原因は「分割」ではなく「平滑化」だと確定した。OpenSubdiv Bilinear（平滑化せず分割するだけの方式）で3回分割しても一辺は  $1.00000$  のまま変わらない。面数は同じ384なので、分割量は同じで平滑化の有無だけが違う。

- ・ OpenSubdiv Loop だけプリミティブ数が  $768$  になる（他は384）。Loop は三角形メッシュ向けの方式なので、四角形が三角形に割られて枚数が倍になる。四角形を保ちたい場面では選んではいけない。

- ・ Mantra-Compatible Catmull-Clark は回数を  $1 \sim 5$  のどれにしても一辺が  $0.83951$  で一定だった。これは分割を繰り返して極限に近づけるのではなく、極限曲面上の位置を直接求めていることを示している。実際この値は、OpenSubdiv CC の収束列から外挿した極限值  $0.83951$  と一致した。つまり「無限回分割した先の形」を1回で出しているのがこのモード。

## 次にやること

- ・ crease（折り目）を使って角を保持できるか。立方体らしさを残したまま分割する方法があるはず

- ・ mountain と subdivide

の順序を入れ替えて、凹凸のディテールがどう変わるかを比較する（実験003）

- ・ polyextrude

など他のSOPでもポイント数・プリミティブ数の増え方を測り、重くなる原因を数で把握する

# mountain と subdivide

## は、順序を変えると何が変わるか

2026-09-13 | Houdini 21.0.700 | /obj/order\_compare | ノード 14 / 接続 13

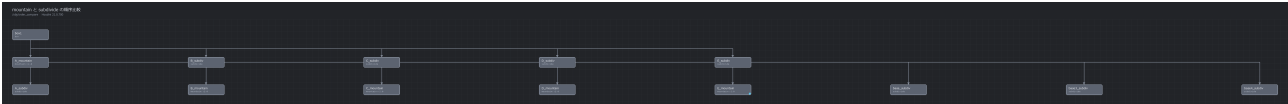
### 何を作ったか

実験001で「mountain はポイントを増やさないの、細かい凹凸を出すには先に分割数を上げる必要がある」と書いた。これは筋は通っているが、実際に確かめてはいなかった。そこで順序違いを並べて比較し、違いを数字で出す。

比較したのは6通り。ノイズなしの基準形、ノイズを先にかけたもの、分割を先にしたもの、分割をさらに細かくしたもの、ノイズの特徴サイズを小さくしたもの、そのうえで点数を増やしたもの。すべて同じカメラでレンダした。

この実験では測り方を一度間違えている。最初に選んだ指標では違いが正しく出ず、途中で別の指標に切り替えた。その経緯も含めて記録した。

### ノードの接続

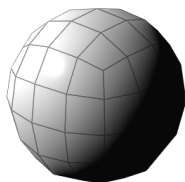


スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

| ノード          | タイプ           | デフォルトから変更したパラメータ                   |
|--------------|---------------|------------------------------------|
| box1         | box           | (変更なし)                             |
| A_mountain   | mountain::2.0 | elementsiz = 0.8<br>height = 0.35  |
| A_subdiv     | subdivide     | iterations = 2                     |
| B_subdiv     | subdivide     | iterations = 2                     |
| B_mountain   | mountain::2.0 | elementsiz = 0.8<br>height = 0.35  |
| C_subdiv     | subdivide     | iterations = 3                     |
| C_mountain   | mountain::2.0 | elementsiz = 0.8<br>height = 0.35  |
| D_subdiv     | subdivide     | iterations = 3                     |
| D_mountain   | mountain::2.0 | elementsiz = 0.25<br>height = 0.35 |
| E_subdiv     | subdivide     | iterations = 4                     |
| E_mountain   | mountain::2.0 | elementsiz = 0.25<br>height = 0.35 |
| base_subdiv  | subdivide     | iterations = 2                     |
| base3_subdiv | subdivide     | iterations = 3                     |
| base4_subdiv | subdivide     | iterations = 4                     |

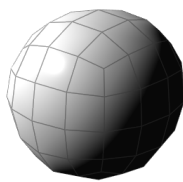
### 6通りの結果

すべて同じカメラ・同じ表示モード。左から右へ、凹凸がどう変わるかを見てほしい。mountainのパラメータは height = 0.35 で固定し、elementsize（ノイズの特徴サイズ）だけを D と E で 0.8 から 0.25 に変えている。



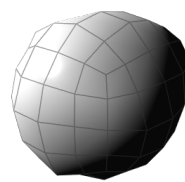
Houdini

基準: 分割のみ、ノイズなし 98点 / 96面



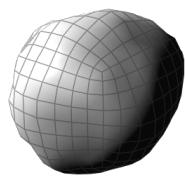
Houdini

A: mountain → subdiv(2) 98点 / 96面



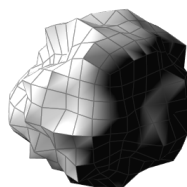
Houdini

B: subdiv(2) → mountain 98点 / 96面



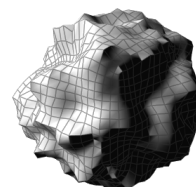
Houdini

C: subdiv(3) → mountain 386点 / 384面



Houdini

D: C と同じだが elementsize 0.25 / 386点



Houdini

E: D と同じノイズで 点数のみ増加 / 1538点

## 基準形からの点ごとのずれ

同じ分割回数ならポイントの順序も同じなので、ノイズなしの基準形と1点ずつ引き算できる。これが「ノイズがどれだけ形を動かしたか」を直接表す。A・B は2回分割の基準形と、C・D は3回分割の基準形と、E は4回分割の基準形と比べている。

| 変種                      | ポイント数 | 平均のずれ   | 最大のずれ   |
|-------------------------|-------|---------|---------|
| A: mountain → subdiv(2) | 98    | 0.02069 | 0.03671 |
| B: subdiv(2) → mountain | 98    | 0.02479 | 0.08062 |
| C: subdiv(3) → mountain | 386   | 0.02504 | 0.08009 |
| D: C + elementsize 0.25 | 386   | 0.03236 | 0.11985 |
| E: D + 点数4倍             | 1538  | 0.03239 | 0.13173 |

## 最初を選んで失敗した指標（表面積）

凹凸が増えれば表面積も増えるはずだと考え、まず表面積を測った。ところが mountain を足すと表面積は減った。ノイズは外側にも内側にも押すため全体が縮み（一辺が 0.8785 から 0.8545

へ)、面積は大きさの2乗に比例するのでその影響が凹凸の効果を打ち消していた。一辺の2乗で割って正規化しても、値の大小が凹凸の量と対応しない。大きさと凹凸が混ざった指標だったということ。

| 変種         | 表面積     | 基準比   | 一辺     | 面積 / 一辺 <sup>2</sup> |
|------------|---------|-------|--------|----------------------|
| 基準 (ノイズなし) | 2.43164 | 1.000 | 0.8785 | 3.15076              |
| A          | 2.30052 | 0.946 | 0.8545 | 3.15066              |
| B          | 2.24947 | 0.925 | 0.8535 | 3.08797              |
| C          | 2.16988 | 0.892 | 0.8314 | 3.13918              |
| D          | 2.49922 | 1.028 | 0.9123 | 3.00282              |
| E          | 2.66892 | 1.098 | 0.9278 | 3.10046              |

## わかったこと

- ・順序を入れ替えてもポイント数とプリミティブ数は同じ (どちらも98点 / 96面)。つまりデータの重さは変わらない。変わるのは形だけ。
- ・最大のずれは A が 0.03671、B が 0.08062 で、B が約2.2倍大きい。同じ mountain の設定でも、分割したあとにかけた方が凹凸が残る。実験001で立てた見込みは正しかった。
- ・なぜそうなるかも測れた。box に mountain をかけた直後は、8点すべてが平均 0.03607 動いている。ところがそのあと subdivide をかけると、基準形からのずれは平均 0.02069・最大 0.03671 まで下がる。Catmull-Clark の平滑化は周囲の点を平均するので、せっかく動かした凹凸をなめしてしまう。
- ・C は B とほぼ同じずれだった (平均 0.02504 対 0.02479、最大 0.08009 対 0.08062)。点を4倍に増やしても凹凸の大きさは変わらない。点数はノイズの振幅を決めていない。
- ・振幅を決めていたのは elementsize だった。C と D は同じ386点だが、elementsiz を 0.8 から 0.25 に下げると平均のずれが 0.02504 から 0.03236 に増え、画も明確に岩ようになった。
- ・E は D と同じノイズ設定で点数だけ4倍にしたもの。平均のずれは 0.03236 から 0.03239 でほぼ変わらない。一方で画は、同じ起伏をより滑らかな面で表現できている。つまり点数は「ノイズをどれだけ忠実に表現できるか (解像度)」を決めていて、「起伏の大きさ (振幅)」は決めていない。
- ・ただし最大のずれだけは点数とともに少し増える (0.11985 → 0.13173)。点が増えるとノイズのより極端な場所まで拾えるようになるため、振幅そのものが増えたわけではない。
- ・実用的な結論: 細かいディテールが欲しいときは、分割数を上げるだけでは足りない。elementsiz も一緒に小さくする。そして mountain は分割のあとに置く。この3つが揃って初めて狙った凹凸になる。
- ・測り方についての反省。表面積のような「全体をひとつの数にまとめた量」は、大きさと形の変化が混ざってしまう。基準形と1点ずつ対応づけて引き算する方が、知りたいことを素直に測れた。比較実験では、まず基準形を作り、それと1対1で比べられる量を選ぶべき。

## 次にやること

- ・mountain の他のパラメータ (roughness, octaves) が振幅と細かさのどちらに効くのかを、同じやり方で切り分ける
- ・grid から地形を作り、平面の場合も同じ結論になるか確かめる (実験004)
- ・crease を使って、分割しても角を保持できるかを確かめる (実験002からの持ち越し)

# mountain

## のノイズパラメータは、それぞれ何に効くのか

2026-09-13 | Houdini 21.0.700 | /obj/mountain\_params | ノード 15 / 接続 14

### 何を作ったか

実験003で「点数は解像度を決め、elementsize は振幅を決める」と分かった。では残りのノイズパラメータは何を変えているのか。同じ切り分け方で調べる。

最初に分かったのは名前の問題だった。パラメータ名は roughness や octaves ではなく rough / oct / lac。実際に mountain::2.0 が持っている名前を確認してから使わないと、そもそも動かない。

この実験でも測り方を追加している。003で使った「基準形からのずれ」だけでは、これらのパラメータをほとんど区別できなかった。振幅ではなく形の性質を変えるパラメータだったため、性質を測る指標を新しく用意した。

### ノードの接続



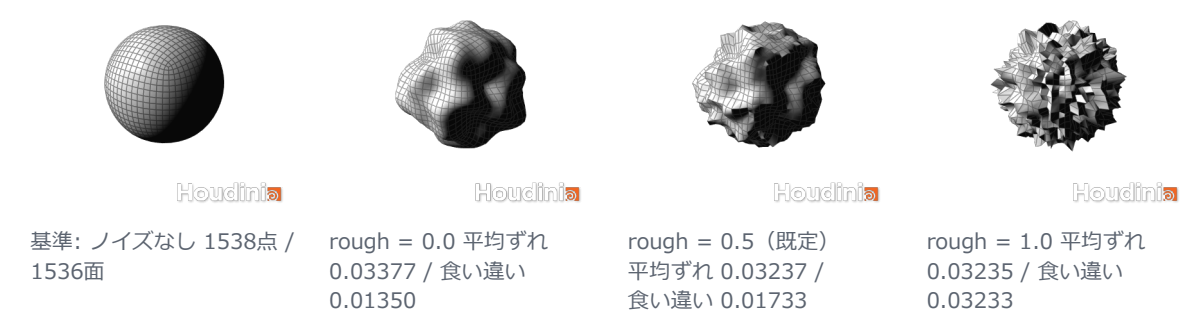
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

| ノード         | タイプ           | デフォルトから変更したパラメータ                                   |
|-------------|---------------|----------------------------------------------------|
| box1        | box           | (変更なし)                                             |
| base_subdiv | subdivide     | iterations = 4                                     |
| rough_00    | mountain::2.0 | elementsize = 0.25<br>height = 0.35<br>rough = 0.0 |
| rough_05    | mountain::2.0 | elementsize = 0.25<br>height = 0.35                |
| rough_10    | mountain::2.0 | elementsize = 0.25<br>height = 0.35<br>rough = 1.0 |
| oct_1       | mountain::2.0 | elementsize = 0.25<br>height = 0.35<br>oct = 1     |
| oct_3       | mountain::2.0 | elementsize = 0.25<br>height = 0.35<br>oct = 3     |
| oct_8       | mountain::2.0 | elementsize = 0.25<br>height = 0.35                |
| lac_15      | mountain::2.0 | elementsize = 0.25<br>height = 0.35<br>lac = 1.5   |
| lac_20      | mountain::2.0 | elementsize = 0.25<br>height = 0.35<br>lac = 2.0   |
| lac_40      | mountain::2.0 | elementsize = 0.25<br>height = 0.35<br>lac = 4.0   |

| ノード          | タイプ           | デフォルトから変更したパラメータ                                       |
|--------------|---------------|--------------------------------------------------------|
| fractal_hmFT | mountain::2.0 | elementsized = 0.25<br>height = 0.35                   |
| fractal_fBm  | mountain::2.0 | elementsized = 0.25<br>fractal = fBm<br>height = 0.35  |
| fractal_mfT  | mountain::2.0 | elementsized = 0.25<br>fractal = mfT<br>height = 0.35  |
| fractal_none | mountain::2.0 | elementsized = 0.25<br>fractal = none<br>height = 0.35 |

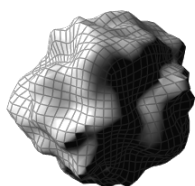
### rough（粗さ） — 振幅は変えず、細かさだけを変える

固定条件はすべての変種で共通: subdivide 4回（1538点）、height 0.35、elementsized 0.25。rough だけを動かしている。平均のずれはほぼ変わらないのに、見た目はなだらかな丘から棘だらけの形まで変わる。



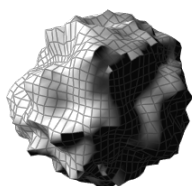
### oct（オクタブ数） — 3 以上で効かなくなる

オクタブとは、細かさの違うノイズを重ねる層のこと。oct はその層の数。1 から 3 では効くが、3 から 8 ではほぼ変わらない（0.01732 → 0.01733）。点数が足りず、細かい層を表現できていないため。



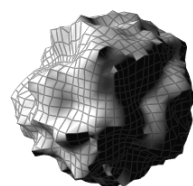
Houdini

oct = 1 食い違い 0.01519



Houdini

oct = 3 食い違い 0.01732



Houdini

oct = 8 (既定) 食い違い 0.01733

| oct    | 平均のずれ   | 最大のずれ   | 隣との食い違い |
|--------|---------|---------|---------|
| 1      | 0.03231 | 0.13417 | 0.01519 |
| 3      | 0.03250 | 0.13274 | 0.01732 |
| 8 (既定) | 0.03237 | 0.13153 | 0.01733 |

### 予測の検証 — oct が効かないのは点数のせいかな

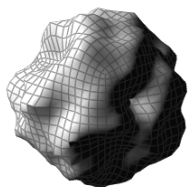
「oct が 3 以上で効かないのは点数が足りないからだ」という説明が正しいなら、点数を増やせば oct=8 が oct=3 を上回るはず。実際にそうなった。386点では oct=8 が oct=3 を下回っているが、6146点では上回っている。

注意: 食い違いの絶対値は点数が増えると小さくなる（隣同士の距離が縮まるので差も縮む）。この表は各行の中で比較だけが有効で、行をまたいで比べてはいけない。

| subdivide | 点数   | oct = 1 | oct = 3 | oct = 8 | 8 と 3 の差 |
|-----------|------|---------|---------|---------|----------|
| 3回        | 386  | 0.02692 | 0.02868 | 0.02856 | -0.00012 |
| 4回        | 1538 | 0.01519 | 0.01732 | 0.01733 | +0.00001 |
| 5回        | 6146 | 0.00782 | 0.00941 | 0.00963 | +0.00022 |

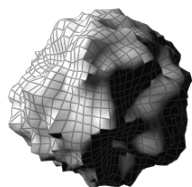
### lac（ラキュナリティ） — オクターブ間の周波数の開き

重ねる層ごとに、どれだけ細かさを飛ばすかの倍率。既定は 1.841。大きくすると層の間隔が開いて細かい成分が強くなる。ただし lac = 1.5 では平均のずれも下がっており、振幅にも影響が出ている。



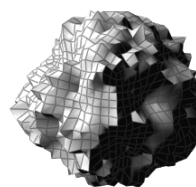
Houdini

lac = 1.5 平均 0.02625 / 食い違い 0.01259



Houdini

lac = 2.0 平均 0.03160 / 食い違い 0.01734



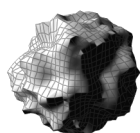
Houdini

lac = 4.0 平均 0.03169 / 食い違い 0.02218

| lac | 平均のずれ   | 最大のずれ   | 隣との食い違い |
|-----|---------|---------|---------|
| 1.5 | 0.02625 | 0.13925 | 0.01259 |
| 2.0 | 0.03160 | 0.13528 | 0.01734 |
| 4.0 | 0.03169 | 0.13334 | 0.02218 |

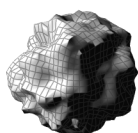
## fractal（フラクタルの種類） — 差は小さい

層の重ね方の方式。4種類あるが、今回の条件では数値の差が小さかった。Terrain だけやや滑らか。既定は Hybrid Terrain。



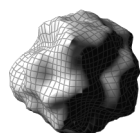
Houdini

Hybrid Terrain（既定）  
食い違い 0.01733



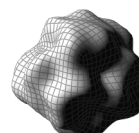
Houdini

Standard (fBm) 食い違い  
0.01786



Houdini

Terrain 食い違い 0.01481



Houdini

None 食い違い 0.01350

| fractal            | 平均のずれ   | 最大のずれ   | 隣との食い違い |
|--------------------|---------|---------|---------|
| Hybrid Terrain（既定） | 0.03237 | 0.13153 | 0.01733 |
| Standard (fBm)     | 0.03183 | 0.13207 | 0.01786 |
| Terrain            | 0.03288 | 0.13504 | 0.01481 |
| None               | 0.03377 | 0.14799 | 0.01350 |

## わかったこと

- ・パラメータ名は rough / oct / lac。roughness や octaves という名前は存在せず、そのまま書くと何も設定されない。既定値は rough = 0.5、oct = 8、lac = 1.841、fractal = Hybrid Terrain、basis = Simplex。使う前に実際の名前を確認する必要がある。
- ・003で使った「基準形からのずれ」だけでは、これらのパラメータはほとんど区別できなかった。平均のずれはどれも 0.032~0.034 の範囲に収まる。振幅を決めているのは height と elementsize であり、今回のパラメータは振幅に効いていない。
- ・そこで「隣り合う点どうして、ずれの向きと量がどれだけ食い違うか」を測る指標を足した。なだらかな起伏なら隣の点も同じように動くので値は小さく、細かい凹凸なら大きくなる。基準形は原点中心なので、中心からの距離の差を符号付きのずれとして使っている（外に出たら正、へこんだら負）。これで完全に分離できた。
- ・rough が細かさに最も強く効く。0.0 から 1.0 で食い違いが 0.01350 から 0.03233 へ約2.4倍になる一方、平均のずれは 0.03377 から 0.03235 でほぼ変わらない。つまり rough は、同じ振幅のまま、なだらかな丘から棘だらけの形まで性質だけを変えるパラメータ。
- ・rough = 0.0 の数値は fractal = None と完全に一致した（平均 0.03377、最大 0.14799、食い違い 0.01350）。rough = 0 では追加の層が何も足さないの、層を重ねない単一ノイズに退化している。別の経路で同じ結果に到達したので、この解釈は確かだと言える。
- ・oct は 1 から 3 では効くが、3 から 8 ではほぼ変わらない（0.01732 → 0.01733、差は 0.00001）。点数が足りず細かい層を表現できていないため。この説明を予測に変えて検証したところ、点数を増やす

と oct = 8 が oct = 3 を上回り始めた（386点では -0.00012、1538点で +0.00001、6146点で +0.00022）。予測どおりだったので、この解釈も確かだと言える。

- ・実用的には、oct を上げるだけでは無駄になる。点数が足りていなければ、上げた層は形に現れない。003の「点数は解像度を決める」という結論と同じことが、別の角度から確認できた。

- ・lac も細かさに効く（1.5 → 4.0 で 0.01259 → 0.02218）。ただし lac = 1.5 では平均のずれも 0.02625 に下がっており、振幅にも影響している。rough のようにきれいに性質だけを変えるパラメータではない。

- ・fractal の4種類は、今回の条件では差が小さかった（食い違い 0.01350~0.01786）。地形向け的方式が揃っているので、平面の地形（実験005の予定）ではもっと差が出る可能性がある。

- ・指標の注意点。食い違いの絶対値は点数が増えると小さくなる。隣同士の距離が縮まれば、その間のずれの差も縮むため。したがって同じ点数の中でだけ比較できる。点数をまたいだ比較には、辺の長さで割るなどの正規化が必要になる。

- ・basis（ノイズの基礎となる種類）は14種類あった。既定は Simplex。Worley Cellular のようなセル状のものも含まれており、見た目が大きく変わるはずなので、別の実験にする価値がある。

## 次にやること

- ・basis を14種類すべて描き出して並べる。セル状ノイズは岩や結晶の見た目に直結するので、実用価値が高い
- ・grid から地形を作り、平面では fractal の種類の差が出るかを確かめる（実験005）
- ・食い違いの指標を辺の長さで正規化し、点数をまたいだ比較ができるようにする

# basis（ノイズの種類） 14通りで、形はどこまで変わるか

2026-09-13 | Houdini 21.0.700 | /obj/noise\_basis | ノード 16 / 接続 15

## 何を作ったか

004で basis に14種類あることが分かったので、同じ条件で全部描き出した。height 0.35、elementsize 0.25、subdivide 4回（1538点）で固定し、basis だけを変えている。

結果は他のパラメータと性質がまったく違った。rough は振幅を変えずに形の性質だけを変えたが、basis は振幅も性質も両方変える。同じ height を指定しても、種類によって実際のずれが4.5倍違う。

測る指標には004で作った「隣との食い違い」に加えて、外に出た点の割合を足した。ノイズが外向きに押すのか内向きに削るのかで、使いどころが変わるため。

## ノードの接続



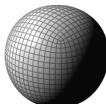
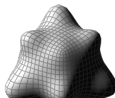
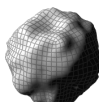
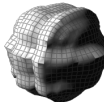
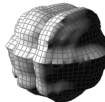
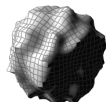
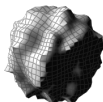
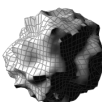
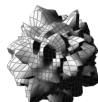
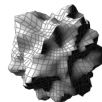
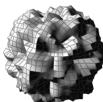
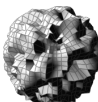
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

| ノード            | タイプ           | デフォルトから変更したパラメータ                                        |
|----------------|---------------|---------------------------------------------------------|
| box1           | box           | (変更なし)                                                  |
| base_subdiv    | subdivide     | iterations = 4                                          |
| basis_sine     | mountain::2.0 | basis = sine<br>elementsize = 0.25<br>height = 0.35     |
| basis_perlin   | mountain::2.0 | basis = perlin<br>elementsize = 0.25<br>height = 0.35   |
| basis_pperlin  | mountain::2.0 | basis = pperlin<br>elementsize = 0.25<br>height = 0.35  |
| basis_simplex  | mountain::2.0 | elementsize = 0.25<br>height = 0.35                     |
| basis_sparse   | mountain::2.0 | basis = sparse<br>elementsize = 0.25<br>height = 0.35   |
| basis_flow     | mountain::2.0 | basis = flow<br>elementsize = 0.25<br>height = 0.35     |
| basis_pflow    | mountain::2.0 | basis = pflow<br>elementsize = 0.25<br>height = 0.35    |
| basis_worleyFA | mountain::2.0 | basis = worleyFA<br>elementsize = 0.25<br>height = 0.35 |
| basis_worleyFB | mountain::2.0 | basis = worleyFB<br>elementsize = 0.25<br>height = 0.35 |

| ノード             | タイプ           | デフォルトから変更したパラメータ                                          |
|-----------------|---------------|-----------------------------------------------------------|
| basis_mworleyFA | mountain::2.0 | basis = mworleyFA<br>elementsized = 0.25<br>height = 0.35 |
| basis_mworleyFB | mountain::2.0 | basis = mworleyFB<br>elementsized = 0.25<br>height = 0.35 |
| basis_cworleyFA | mountain::2.0 | basis = cworleyFA<br>elementsized = 0.25<br>height = 0.35 |
| basis_cworleyFB | mountain::2.0 | basis = cworleyFB<br>elementsized = 0.25<br>height = 0.35 |
| basis_alligator | mountain::2.0 | basis = alligator<br>elementsized = 0.25<br>height = 0.35 |

### 14種類すべて（細かさの小さい順）

すべて同じカメラ・同じ設定。左上のなめらかなものから、右下の硬く割れたものへ並んでいる。

|   |   |   |   |   |          |
|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|----------|
| Houdini                                                                             | Houdini                                                                             | Houdini                                                                             | Houdini                                                                              | Houdini                                                                               |          |
| 基準: ノイズなし<br>1538点                                                                  | Sinusoid 平均<br>0.14454 / 細かさ<br>0.00444                                             | Sparse Convolution<br>平均 0.01985 /<br>細かさ 0.00965                                   | Periodic Perlin Flow<br>平均 0.01551 /<br>細かさ 0.01005                                  | Periodic Perlin 平均<br>0.01662 / 細かさ<br>0.01062                                        |          |
|  |  |  |  |  |          |
| Houdini                                                                             | Houdini                                                                             | Houdini                                                                             | Houdini                                                                              | Houdini                                                                               |          |
| Perlin 平均 0.02426<br>/ 細かさ 0.01268                                                  | Perlin Flow 平均<br>0.02639 / 細かさ<br>0.01441                                          | Simplex (既定)<br>平均 0.03237 /<br>細かさ 0.01733                                         | Worley F2-F1 平均<br>0.14144 / 細かさ<br>0.01831                                          | Worley F1 平均<br>0.08017 / 細かさ<br>0.02065                                              |          |
|  |  |  |  |  |          |
| Houdini                                                                             | Houdini                                                                             | Houdini                                                                             | Houdini                                                                              | Houdini                                                                               |          |
| Alligator 平均<br>0.12904 / 細かさ<br>0.02747                                            | Manhattan F1 平均<br>0.08103 / 細かさ<br>0.02996                                         | Chebyshev F2-F1<br>平均 0.11841 /<br>細かさ 0.03104                                      | Chebyshev F1 平均<br>0.09937 / 細かさ<br>0.03193                                          | Manhattan F2-F1<br>平均 0.11505 /<br>細かさ 0.03323                                        |          |
| basis                                                                               | 表示名                                                                                 | 平均のずれ                                                                               | 最大のずれ                                                                                | 隣との食い違い                                                                               | 外向きの点の割合 |
| sine                                                                                | Sinusoid                                                                            | 0.14454                                                                             | 0.15748                                                                              | 0.00444                                                                               | 0.0%     |

| basis     | 表示名                      | 平均のずれ   | 最大のずれ   | 隣との食い違い | 外向きの点の割合 |
|-----------|--------------------------|---------|---------|---------|----------|
| sparse    | Sparse Convolution       | 0.01985 | 0.08378 | 0.00965 | 32.9%    |
| pflow     | Periodic Perlin Flow     | 0.01551 | 0.05039 | 0.01005 | 51.2%    |
| pperlin   | Periodic Perlin          | 0.01662 | 0.05705 | 0.01062 | 58.8%    |
| perlin    | Perlin                   | 0.02426 | 0.09921 | 0.01268 | 40.9%    |
| flow      | Perlin Flow              | 0.02639 | 0.10496 | 0.01441 | 49.9%    |
| simplex   | Simplex (既定)             | 0.03237 | 0.13153 | 0.01733 | 35.8%    |
| worleyFB  | Worley Cellular F2-F1    | 0.14144 | 0.17436 | 0.01831 | 0.0%     |
| worleyFA  | Worley Cellular F1       | 0.08017 | 0.17328 | 0.02065 | 5.4%     |
| alligator | Alligator                | 0.12904 | 0.17456 | 0.02747 | 2.3%     |
| mworleyFA | Manhattan Cellular F1    | 0.08103 | 0.16473 | 0.02996 | 91.1%    |
| cworleyFB | Chebyshev Cellular F2-F1 | 0.11841 | 0.17439 | 0.03104 | 1.4%     |
| cworleyFA | Chebyshev Cellular F1    | 0.09937 | 0.16540 | 0.03193 | 94.4%    |
| mworleyFB | Manhattan Cellular F2-F1 | 0.11505 | 0.17052 | 0.03323 | 2.5%     |

## わかったこと

- basis は振幅と性質の両方を変える。rough

が振幅を保ったまま性質だけを変えたのとは性格が違う。同じ height 0.35 でも、平均のずれは Periodic Perlin Flow の 0.01551 から Sinusoid の 0.14454 まで約9.3倍の開きがある。basis を変えたら height も再調整が必要。

- Sinusoid は例外的な挙動をする。平均のずれが 0.14454 と最大級に大きいのに、隣との食い違いは 0.00444 で全種類中もっとも小さい。外向きの点は 0.0%、つまり全部の点が内側に動いている。画を見ると、丸い4つの膨らみを持つ滑らかな形になっていた。規則的な波なので、細かい凹凸ではなく大きなうねりを作る。地形の大きな起伏には使えるが、ディテールには使えない。

- Perlin 系 (perlin / pperlin / flow / pflow) は外向きの点が 40~59% で、外と内にほぼ均等に動く。平均のずれも 0.0155~0.0264 と控えめ。自然な揺らぎを足したいときの素直な選択肢。

- 既定の Simplex は Perlin 系より振幅も細かさも大きい (平均 0.03237、細かさ 0.01733) が、セル系ほど極端ではない。中間的な位置にある。

- セル系 (Worley / Manhattan / Chebyshev / Alligator) は平均のずれが 0.08~0.14 と大きく、隣との食い違いも上位を占める。岩、割れ、結晶のような硬い見た目はここから作る。

- セル系の中で、外向きの点の割合が極端に分かれた。Manhattan F1 が 91.1%、Chebyshev F1 が 94.4% で、ほぼ全部の点が外に出る。一方 Worley F1 は 5.4%、Alligator は 2.3%、Chebyshev F2-F1 は 1.4% で、ほぼ全部が内側に削られる。

- ・画で確認すると理由が見えた。Worley F1

は、細胞の中心から遠いところが深く削られた侵食岩のような形になる。Manhattan F1 は距離の測り方が軸に沿っているため、軸に平行な面を持つ角ばった塊が外に突き出す形になった。同じ「F1」でも距離の定義が違うと、削る方向と形がまったく変わる。

・ F1 と F2-F1 の違いも外向き率に出る。Manhattan は F1 が 91.1% (外向き) なのに F2-F1 は 2.5% (内向き)。Chebyshev も F1 が 94.4%、F2-F1 が 1.4% で同じ傾向。F2-F1 は細胞の境界線を強調する計算なので、境界以外が下がる = 内向きになる。

・最大のずれはセル系でほぼ頭打ちになっている (0.164~0.175)。height 0.35 に対して 0.5 倍弱で止まっており、何らかの上限が効いている可能性がある。この確認は別の実験にする。

・外向きの点の割合という指標は安く作れて情報量が多かった。ノイズが押すのか削るのかは、実際に作りたいもの (突起か窪みか) に直結する。

## 次にやること

- ・セル系で最大のずれが 0.175 付近で止まる理由を確かめる。height を上げても止まるのかを測る
- ・grid の平面で同じ14種類を並べる。地形として見たときの向き不向きは球体とは違うはず
- ・外向き率を、突起を作りたいとき／窪みを作りたいときの選択指針としてまとめる

# height は純粋な倍率か。gain と bias は何に効くか

2026-09-13 | Houdini 21.0.700 | /obj/height\_shape | ノード 10 / 接続 9

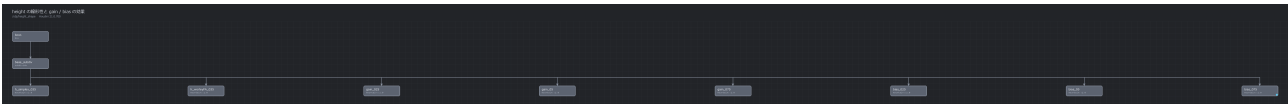
## 何を作ったか

005でセル系の最大のずれが 0.164~0.175  
に揃っていたので、何らかの上限が効いているのではないかと書いた。それを確かめる実験。height を 0.0875 から 1.4 まで16倍の範囲で変え、最大のずれが比例するかを見る。

結論を先に書くと、005の見立ては誤りだった。上限は存在しない。そして高い height で比率が崩れたのは Houdini の制限ではなく、私が使っている指標が壊れていたためだった。

あわせて未調査だった gain / bias / clipmin / clipmax も測った。bias は実用上かなり重要なパラメータだと分かった。

## ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

| ノード            | タイプ           | デフォルトから変更したパラメータ                                                 |
|----------------|---------------|------------------------------------------------------------------|
| box1           | box           | (変更なし)                                                           |
| base_subdiv    | subdivide     | iterations = 4                                                   |
| h_simplex_035  | mountain::2.0 | elementsize = 0.25<br>height = 0.35                              |
| h_worleyFA_035 | mountain::2.0 | basis = worleyFA<br>elementsize = 0.25<br>height = 0.35          |
| gain_025       | mountain::2.0 | dogain = 1<br>elementsize = 0.25<br>gain = 0.25<br>height = 0.35 |
| gain_05        | mountain::2.0 | dogain = 1<br>elementsize = 0.25<br>height = 0.35                |
| gain_075       | mountain::2.0 | dogain = 1<br>elementsize = 0.25<br>gain = 0.75<br>height = 0.35 |
| bias_025       | mountain::2.0 | bias = 0.25<br>dobias = 1<br>elementsize = 0.25<br>height = 0.35 |
| bias_05        | mountain::2.0 | dobias = 1<br>elementsize = 0.25<br>height = 0.35                |
| bias_075       | mountain::2.0 | bias = 0.75<br>dobias = 1<br>elementsize = 0.25<br>height = 0.35 |

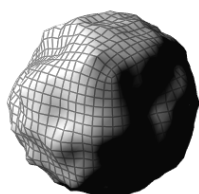
## height を変えたときの最大のずれ

固定条件: subdivide 4回 (1538点)、elementsize 0.25。「最大 / height」が一定なら、height は単なる倍率ということになる。

| basis    | height | 平均のずれ   | 最大のずれ   | 最大 / height | 隣との食い違い |
|----------|--------|---------|---------|-------------|---------|
| simplex  | 0.0875 | 0.00809 | 0.03289 | 0.37594     | 0.00433 |
| simplex  | 0.1750 | 0.01619 | 0.06578 | 0.37590     | 0.00867 |
| simplex  | 0.3500 | 0.03237 | 0.13153 | 0.37579     | 0.01733 |
| simplex  | 0.7000 | 0.06473 | 0.26273 | 0.37532     | 0.03466 |
| simplex  | 1.4000 | 0.12822 | 0.43122 | 0.30801     | 0.06868 |
| worleyFA | 0.0875 | 0.02005 | 0.04332 | 0.49513     | 0.00516 |
| worleyFA | 0.1750 | 0.04009 | 0.08665 | 0.49512     | 0.01033 |
| worleyFA | 0.3500 | 0.08017 | 0.17328 | 0.49508     | 0.02065 |
| worleyFA | 0.7000 | 0.16025 | 0.34636 | 0.49480     | 0.04125 |
| worleyFA | 1.4000 | 0.26687 | 0.42980 | 0.30700     | 0.07338 |

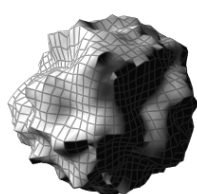
## gain — 振幅の倍率。外向きと内向きの比率は変えない

gain は dogain を有効にしないと効かない（既定は無効）。gain = 0.5 のときの数値は、dogain を無効にした既定状態と完全に一致した。つまり 0.5 が中立値。



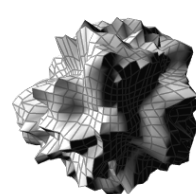
Houdini

gain 0.25 平均 0.01494 / 外向き 35.8%



Houdini

gain 0.50 (中立) 平均 0.03237 / 外向き 35.8%



Houdini

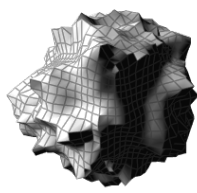
gain 0.75 平均 0.05519 / 外向き 35.8%

| gain      | 平均のずれ   | 最大のずれ   | 隣との食い違い | 外向きの点の割合 |
|-----------|---------|---------|---------|----------|
| 0.25      | 0.01494 | 0.07691 | 0.00816 | 35.8%    |
| 0.50 (中立) | 0.03237 | 0.13153 | 0.01733 | 35.8%    |
| 0.75      | 0.05519 | 0.16402 | 0.02870 | 35.8%    |

## bias — 外向きと内向きの比率を直接変える

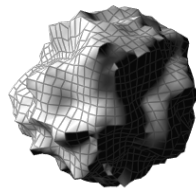
bias も dobias を有効にしないと効かない。0.5 が中立。0.25 にするとほぼ全部が内側に削られ、0.75 にするとほぼ全部が外側に膨らむ。005では basis

を選ぶことでしか得られなかった「突起か窪みか」の制御が、1つのパラメータで連続的にできる。



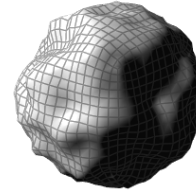
Houdini

bias 0.25 外向き 0.9% =  
ほぼ全部が窪み



Houdini

bias 0.50 (中立) 外向き 35.8%



Houdini

bias 0.75 外向き 98.7% =  
ほぼ全部が膨らみ

| bias      | 平均のずれ   | 最大のずれ   | 隣との食い違い | 外向きの点の割合 |
|-----------|---------|---------|---------|----------|
| 0.25      | 0.09756 | 0.16935 | 0.01614 | 0.9%     |
| 0.50 (中立) | 0.03237 | 0.13153 | 0.01733 | 35.8%    |
| 0.75      | 0.07644 | 0.14054 | 0.01156 | 98.7%    |

## わかったこと

- ・ height は純粋な倍率だった。「最大 / height」は 0.0875 から 0.7

まで8倍の範囲で小数点以下4桁まで一定 (simplex は 0.3759、worleyFA は 0.4951)。height を2倍にすれば、ずれもきっちり2倍になる。

- ・ 005で疑った上限は存在しなかった。セル系の最大のずれが揃って見えたのは、セル系ノイズのピーク値の分布が互いに似ているため。実際 005 のセル系7種類の「最大 / height」を計算すると 0.47~0.50 の範囲に収まる。一方 simplex は 0.376

で明確に違う。上限ではなく、ノイズの種類ごとの固有の比率だった。

- ・ ただし height = 1.4 では比率が両方とも 0.307 に落ちた。これは Houdini の制限ではなく、私の指標が壊れているため。基準形の半径は約 0.42 (subdivide 4回のバウンディングボックスの一辺 0.84192 の半分) で、そのとき測れた最大のずれ 0.4312 / 0.4298 はこれとほぼ一致している。中心を貫通するほど強く押し込まれると、「中心からの距離の差」という測り方では値が頭打ちになる。

- ・ つまりこの指標には適用範囲がある。ずれが対象の半径より十分小さいときにしか使えない。今回でいえば height 0.7 まで。005の観察を「上限がある」と解釈しかけたのは、この限界を意識していなかったため。指標を作ったら、それが壊れる条件も押さえておく必要がある。

- ・ clipmin / clipmax の既定は 0.0 / 1.0、dogain / dobias は既定で無効。つまり既定状態ではクリップもゲインもバイアスも効いていない。005までの測定に余計な処理は入っていなかったことが確認できた。

- ・ gain は振幅の倍率として働く。0.25 / 0.5 / 0.75 で平均のずれが 0.01494 / 0.03237 / 0.05519 と増える。一方で外向きの点の割合は 35.8% のまま完全に動かない。形の偏りには触れず、強さだけを変えるパラメータ。

- ・ gain = 0.5 の数値は、dogain を無効にした既定状態と完全に一致した (平均 0.03237、最大 0.13153、細かさ 0.01733)。0.5 が中立値だと確認できた。

- ・ bias は外向きと内向きの比率を直接変える。0.25 で外向き 0.9% (ほぼ全部が内側に削られる)、0.5 で 35.8%、0.75 で 98.7% (ほぼ全部が外側に膨らむ)。画で見ると、0.25 は深くえぐられて鋭い稜線が残る侵食的な形、0.75 は膨らんで浅いへこみが残る塊になる。

- ・ただし bias を 0.5 から動かすと平均のずれも増える (0.03237 → 0.09756 / 0.07644) 。分布が片側に寄ると、ゼロから離れた点が増えるため。純粋に形の偏りだけを変えるパラメータではなく、振幅も一緒に動く。

- ・ここまでで mountain のパラメータ地図がほぼ埋まった。振幅を決めるのは height (純粋な倍率) と gain (倍率) 。細かさを決めるのは rough (最も強い) 、lac、oct (点数が足りている範囲で) 。外向きか内向きかを決めるのは bias (連続的) と basis (種類で決まる) 。そして何を表現できるかの上限は点数が決める。

## 次にやること

- ・ grid の平面で同じ測定をする。球体では中心からの距離で測れたが、平面では別の基準が必要になるので、指標の作り直しも兼ねる

- ・ 指標を辺の長さで正規化し、点数をまたいだ比較ができるようにする

- ・ パラメータ調査はここで区切り、scatter と copy to points に進む

# 平面で地形を作り、指標を作り直す

2026-09-13 | Houdini 21.0.700 | /obj/grid\_terrain | ノード 12 / 接続 8

## 何を作ったか

これまでの指標は「中心からの距離の差」で、球体でしか使えず、点数をまたいだ比較もできなかった（006でその限界にぶつかった）。平面なら高さの差をそのまま測れる。さらに点の間隔で割れば勾配（傾き）になるので、これなら解像度によらない量になるはずだと考えた。

結果として、その期待は外れた。勾配は解像度を上げるほど増え続ける。ただしこれは失敗ではなく、狙っていたものより役に立つ道具になった。勾配が増えなくなる点が「その形を表現するのに十分な解像度」を教えてくれる。

あわせて、004で「平面ならもっと差が出るかもしれない」と書いた fractal の種類も確かめた。

## ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

| ノード       | タイプ           | デフォルトから変更したパラメータ                    |
|-----------|---------------|-------------------------------------|
| grid_20   | grid          | cols = 20<br>rows = 20              |
| mt_20     | mountain::2.0 | elementsize = 2.0                   |
| grid_40   | grid          | cols = 40<br>rows = 40              |
| mt_40     | mountain::2.0 | elementsize = 2.0                   |
| grid_80   | grid          | cols = 80<br>rows = 80              |
| mt_80     | mountain::2.0 | elementsize = 2.0                   |
| grid_160  | grid          | cols = 160<br>rows = 160            |
| mt_160    | mountain::2.0 | elementsize = 2.0                   |
| frac_none | mountain::2.0 | elementsize = 2.0<br>fractal = none |
| frac_fBm  | mountain::2.0 | elementsize = 2.0<br>fractal = fBm  |
| frac_mfT  | mountain::2.0 | elementsize = 2.0<br>fractal = mfT  |
| frac_hmfT | mountain::2.0 | elementsize = 2.0                   |

## 解像度を変える（elementsize 2.0、height 1.0 固定）

10×10 の grid の分割数を 20 から 160 まで上げた。点の間隔はそれに応じて 0.53 から 0.06 まで細くなる。

| 分割数 | 点数  | 点の間隔   | 平均の高さ   | 最大の高さ   | 平均勾配    |
|-----|-----|--------|---------|---------|---------|
| 20  | 400 | 0.5263 | 0.07919 | 0.29407 | 0.13595 |

| 分割数 | 点数    | 点の間隔   | 平均の高さ   | 最大の高さ   | 平均勾配    |
|-----|-------|--------|---------|---------|---------|
| 40  | 1600  | 0.2564 | 0.08187 | 0.32680 | 0.16634 |
| 80  | 6400  | 0.1266 | 0.08183 | 0.32995 | 0.18745 |
| 160 | 25600 | 0.0629 | 0.08196 | 0.33513 | 0.20430 |

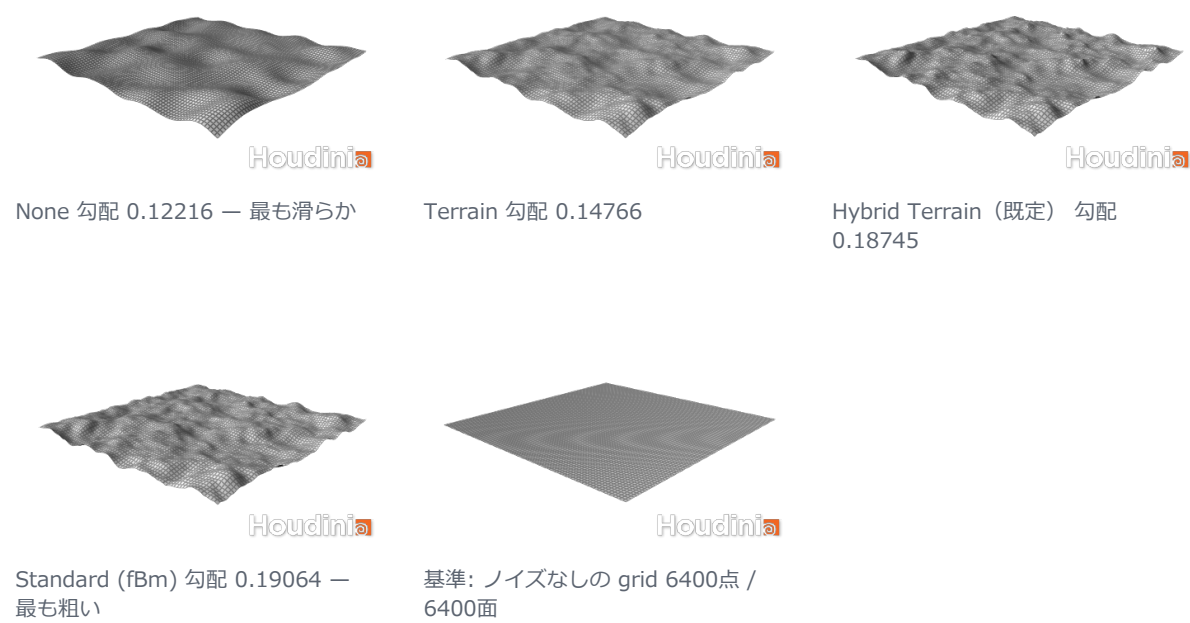
### オクターブ数ごとに、勾配がどこで止まるか

各行が1つの解像度。オクターブ数を増やすほど細かい成分が入るので、それを表現しきるのに必要な解像度も上がる。数字が横に並んで止まっていれば、その解像度で足りているということ。

| 分割数   | oct = 1    | oct = 2      | oct = 4    | oct = 8    | oct=8 の<br>80→160 増加率 |
|-------|------------|--------------|------------|------------|-----------------------|
| 20    | 0.12897    | 0.13673      | 0.13593    | 0.13595    | —                     |
| 40    | 0.14205    | 0.15845      | 0.16659    | 0.16634    | —                     |
| 80    | 0.14492    | 0.16561      | 0.18604    | 0.18745    | —                     |
| 160   | 0.14570    | 0.16752      | 0.19605    | 0.20430    | +9.0%                 |
| 収束したか | した (+0.5%) | ほぼした (+1.2%) | まだ (+5.4%) | まだ (+9.0%) |                       |

### fractal の4種類（分割数80）

004では球体で比べて差が小さかった（細かさ 0.01350~0.01786）。平面では勾配 0.12216 から 0.19064 まで1.6倍の開きが出た。004で立てた「平面ならもっと差が出るかもしれない」という見込みは当たっていた。



| fractal | 平均の高さ   | 最大の高さ   | 平均勾配    | 上向きの点の割合 |
|---------|---------|---------|---------|----------|
| None    | 0.08386 | 0.39362 | 0.12216 | 44.4%    |
| Terrain | 0.08257 | 0.38162 | 0.14766 | 43.6%    |

| fractal             | 平均の高さ   | 最大の高さ   | 平均勾配    | 上向きの点の割合 |
|---------------------|---------|---------|---------|----------|
| Hybrid Terrain (既定) | 0.08183 | 0.32995 | 0.18745 | 42.3%    |
| Standard (fBm)      | 0.08269 | 0.37818 | 0.19064 | 45.4%    |

## わかったこと

- ・ grid SOP の既定は 10×10 のサイズ、分割は 10×10、向きは ZX 平面（水平）、種類はポリゴン。水平に置かれるので、mountain の押し出しは真上（Y方向）になる。だから高さの差をそのまま測れる。
- ・ 狙いは外れた。勾配は解像度によらない量にはならず、分割数を上げるほど増え続けた（0.13595 → 0.16634 → 0.18745 → 0.20430）。fBm 系のノイズは細かい成分を何段も持っているので、細かく刻むほど急な斜面が新たに見えてくるため。
- ・ ただしこの性質のおかげで、別のことが分かるようになった。オクターブ数ごとに勾配の伸びを見ると、oct = 1 は 80分割で収束し（80→160 で +0.5%）、oct = 2 はほぼ収束（+1.2%）、oct = 4 はまだ伸び（+5.4%）、oct = 8 もまだ伸びている（+9.0%）。つまり勾配が増えなくなったら、その解像度でその形を表現しきれているという判定に使える。
- ・ 低い解像度では余分なオクターブがまったく見えない。20分割では oct = 2 / 4 / 8 がほぼ同じ値（0.13673 / 0.13593 / 0.13595）になり、oct = 2 がわずかに最大という順序の逆転まで起きている。004で球体を使って得た結論が平面でも再現し、しかも解像度との関係まで数字で見えた。
- ・ 高さの平均は40分割以降ほぼ一定（0.0819前後）。振幅は低い解像度でもすぐ正しくなるが、細かさはずっと遅れて収束する。形が正しく見えるのに必要な解像度は、高さが合う解像度よりはるかに高いということ。点数を決めるときは高さではなく細かさを基準にする必要がある。
- ・ 最大の高さも解像度とともに増える（0.29407 → 0.33513）。これも同じ理由で、細かく刻むほどノイズの尖った部分を拾えるようになるため。005・006で使った「最大のずれ」も同じ性質を持っていたことになる。
- ・ fractal の種類は平面で明確に差が出た。滑らかな順に None（0.12216）、Terrain（0.14766）、Hybrid Terrain（0.18745）、Standard fBm（0.19064）。一方で平均の高さはどれも 0.0818~0.0839 でほぼ同じ。fractal は振幅ではなく細かさを変えるパラメータだと確定した。
- ・ None が最も滑らかなのは筋が通っている。層を重ねない単一のノイズなので、細かい成分が入らない。004で「rough = 0 は fractal = None と完全に一致する」と分かったことと整合する。
- ・ カメラのフレーミングを直した。以前はバウンディングボックスを立方体とみなして距離を決めていたため、平たい地形が画面の4分の1ほどにしか写らなかった。8つの隅をカメラ空間に投影して必要な距離を解く方式に変更し、対象が画面いっぱい収まるようになった。地形を扱う以上、これは必須の修正だった。

## 次にやること

- ・ elementsize・オクターブ数・必要な分割数の関係を式にまとめる。「この設定ならこれだけの点数が必要」と事前に見積もれるようにする
- ・ 地形専用の heightfield ノード群と、この grid + mountain のやり方を比べる
- ・ パラメータの調査はここで区切り、scatter と copy to points に進む

## scatter と copy to points — 地面に物を生やす

2026-09-13 | Houdini 21.0.700 | /obj/scatter\_copy | ノード 12 / 接続 13

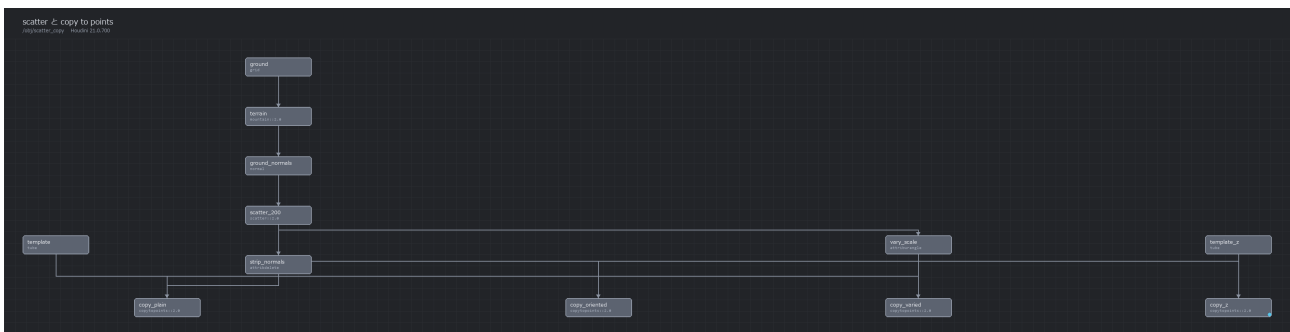
## 何を作ったか

面の上に点をばらまき、その点ごとにジオメトリを複製する。プロシージャルモデリングの中核パターンで、草や岩や建物を並べるのは全部これで行う。

測ったのは3つ。散布点の数は指定どおりになるか、複製後の点数はどう増えるか、そして複製が地面の傾きに沿って回るかどうか。

3つめで重要な落とし穴が見つかった。複製は確かに回るのだが、回り方が想像と違う。数字で確定させた。

## ノードの接続



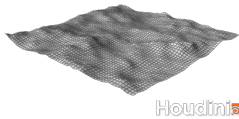
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

| ノード            | タイプ               | デフォルトから変更したパラメータ                                                              |
|----------------|-------------------|-------------------------------------------------------------------------------|
| ground         | grid              | cols = 60<br>rows = 60                                                        |
| terrain        | mountain::2.0     | elementsiz = 3.0<br>height = 1.2                                              |
| ground_normals | normal            | (変更なし)                                                                        |
| scatter_200    | scatter::2.0      | npts = 200                                                                    |
| template       | tube              | cap = 1<br>cols = 10<br>height = 0.9<br>rad1 = 0.0<br>rad2 = 0.18<br>type = 1 |
| strip_normals  | attribdelete      | ptdel = N                                                                     |
| copy_plain     | copytopoints::2.0 | (変更なし)                                                                        |
| copy_oriented  | copytopoints::2.0 | (変更なし)                                                                        |
| vary_scale     | attribwrangle     | snippet = @pscale = fit01(rand(@ptnum * 7.3), 0.35, 1.4);                     |
| copy_varied    | copytopoints::2.0 | (変更なし)                                                                        |

| ノード        | タイプ               | デフォルトから変更したパラメータ                                                                            |
|------------|-------------------|---------------------------------------------------------------------------------------------|
| template_z | tube              | cap = 1<br>cols = 10<br>height = 0.9<br>orient = 2<br>rad1 = 0.0<br>rad2 = 0.18<br>type = 1 |
| copy_z     | copytopoints::2.0 | (変更なし)                                                                                      |

### 複製の向きと大きさ

地面は 10×10 の grid（60分割）に mountain をかけたもの。散布点は200個、テンプレートは20点の円錐。3つの条件で複製した。



地面 grid 60分割 + mountain



法線なしの点に複製  
向きのばらつき 0.00度



法線ありの点に複製 + pscale 向きのばらつき 10.84度



テンプレートをZ軸向きにした場合 N と完全に一致する

| 条件            | 複製後の点数 | 複製数 | 予想点数 | 向きのばらつき（平均 / 最大） | 大きさの幅       |
|---------------|--------|-----|------|------------------|-------------|
| 法線なし          | 4000   | 200 | 4000 | 0.00度 / 0.00度    | 0.45 固定     |
| 法線あり          | 4000   | 200 | 4000 | 10.84度 / 31.69度  | 0.45 固定     |
| 法線あり + pscale | 4000   | 200 | 4000 | 10.84度 / 31.69度  | 0.16 ~ 0.63 |

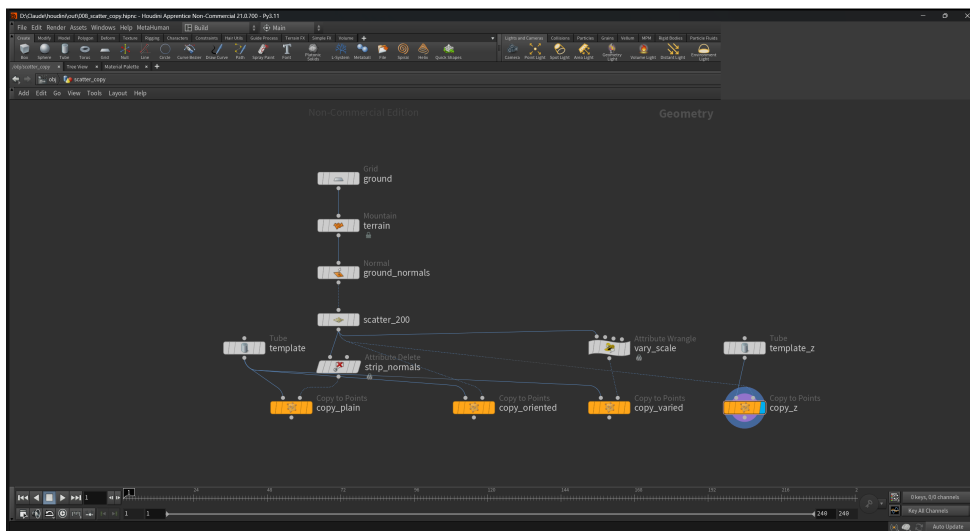
### N はテンプレートのどの軸に対応するのか

複製が地面に沿って回ってはいるのに、円錐が横倒しに見えた。そこで、各複製の軸と、その複製が乗っている点の N との角度を全200個について測った。結果はばらつきゼロで、完全に一方の答えを示した。

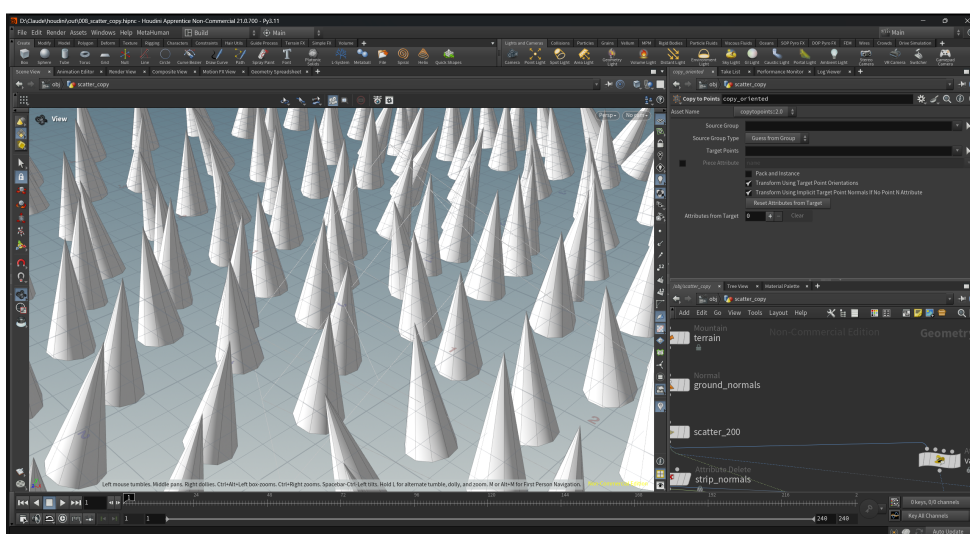
| テンプレートの向き    | N との角度（平均） | 最小     | 最大     |
|--------------|------------|--------|--------|
| Y軸（tube の既定） | 90.00度     | 90.00度 | 90.00度 |
| Z軸           | 0.00度      | 0.00度  | 0.00度  |

### 実際の Houdini の画面

自動生成の接続図と違い、これは起動中の Houdini を外から操作して撮った本物の画面。自分の Houdini で同じものを開いたときに何が見えるかがそのまま分かる。パラメータ側に写っている「Transform Using Target Point Orientations」が、この実験で測った「N で回転するかどうか」の正体。



ネットワークエディタ。scatter\_200 から4つの Copy to Points に分岐している



copy\_oriented のパラメータと、その結果のビューポート

## scatter の点数は指定どおりか

| 指定した数 | 実際の点数 | 一致 |
|-------|-------|----|
| 50    | 50    | はい |
| 200   | 200   | はい |
| 800   | 800   | はい |

## わかったこと

- ・ scatter の npts は正確に守られる。50 / 200 / 800 を指定して、実際の点数も 50 / 200 / 800 になった。およその数ではなく、指定した数ちょうどが出る。
- ・ scatter の出力は P だけでなく N も持っている。元の面の法線が引き継がれるため。この N があるかないかで、次の copy to points の挙動が変わる。
- ・ copy to points の点数は掛け算そのまま。散布点200個 × テンプレート20点 = 4000点。予想と実測が完全に一致した。テンプレートが重いと一気に破綻するので、散布数とテンプレートの点数は掛け算で効くと覚えておく。

- ・ N を持たない点に複製すると、複製は一切回転しない。向きのばらつきは平均も最大も 0.00度。200個すべてが同じ向きに立つ。

- ・ N を持つ点に複製すると、点ごとに回転する。向きのばらつきは平均 10.84度、最大 31.69度で、これは地面の傾きのばらつきそのもの。

- ・ ただし N はテンプレートの Y 軸ではなく Z 軸に対応している。tube の既定 (Y軸向き) のまま複製すると、複製の軸と N の角度が全200個すべて正確に 90.00度、つまり完全に横倒しになる。テンプレートを Z軸向きに作り直すと全200個すべて 0.00度で完全に一致した。ばらつきがゼロなので、これは偶然ではなく仕様。

- ・ tube SOP の orient の既定は Y軸。そのため何も考えずに tube を複製すると必ず横倒しになる。地面に立てたいなら、テンプレートを Z軸向きに作るか、あとで回転させる必要がある。

- ・ pscale は大きさだけを変える。向きのばらつきは 10.84度 / 31.69度 のまま完全に同じで、大きさが 0.45 固定から 0.16~0.63 に広がった。向きと大きさは独立して制御できる。

- ・ tube SOP の type の既定は Primitive で、1点 / 1面の「本物の円柱プリミティブ」になる。ポリゴンとして扱いたいなら Polygon に変える必要がある。これに気づかず、最初は1点のテンプレートを200個複製していた (200点になるだけで何も見えない) 。

- ・ 測り方の失敗も1つ記録しておく。最初は「重心から最も遠い点」で複製の向きを測ろうとしたが、円錐の底面リングの点がすべて等距離のため、選ばれる点が複製ごとにばらついて 21.44度 という無意味な値が出た。固定の点番号を使う方式に変えたら 0.00度 になった。等距離の点が並ぶ形では「最も遠い点」は一意に決まらない。

- ・ 実用の型はこうなる。地面 → normal → scatter → copy to points。テンプレートは Z軸向きに作る。大きさを変えたいなら散布点に pscale を持たせる。この4ノードで「地面に物を生やす」が完成する。

## 次にやること

- ・ 複製の向きに、傾きだけでなくランダムな回転を混ぜる (up ベクトルの扱い)
- ・ density や面積のアトリビュートで散布密度に偏りをつける
- ・ for-each で複製ごとに違う処理をかける (実験010の予定)

# attribute wrangle で VEX を書く ー 何が何回動くのか

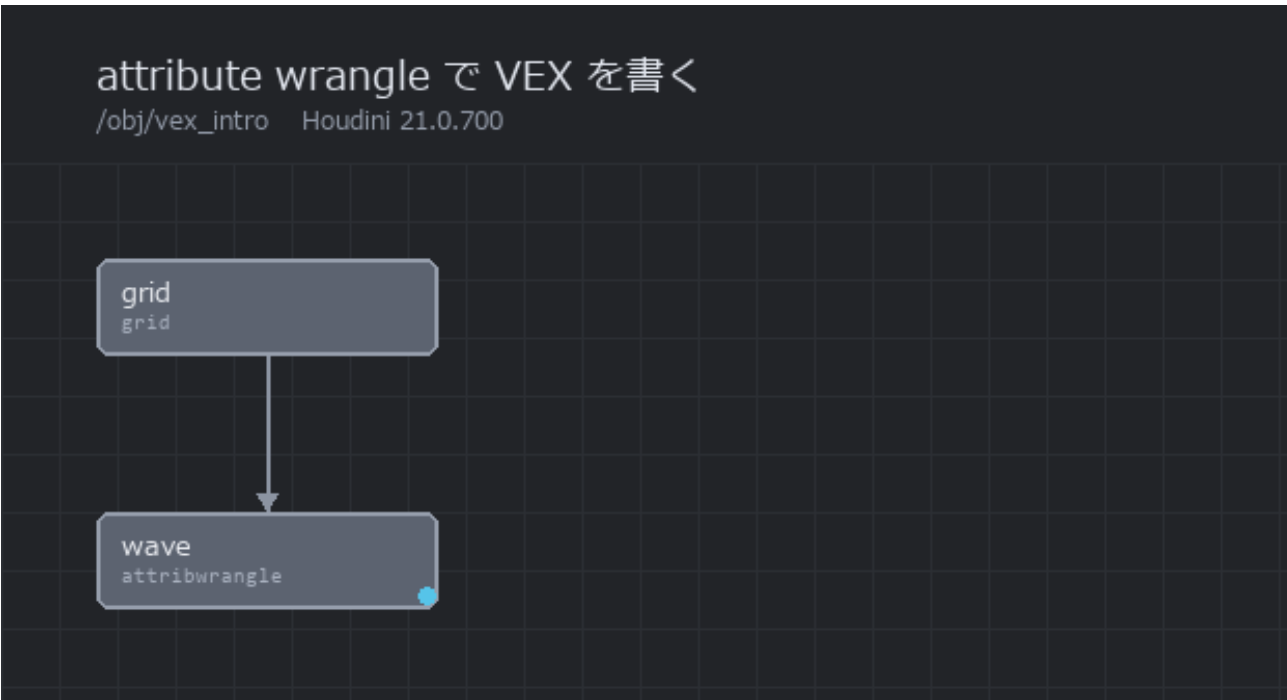
2026-09-13 | Houdini 21.0.700 | /obj/vex\_intro | ノード 2 / 接続 1

## 何を作ったか

VEX は「ノードを並べる代わりに数行書く」ための言語。ただし初心者が最初につまずくのは文法ではなく、そのコードが何に対して何回走るのかという点。それを実測で確かめる。

あわせて「VEXは速い」という話を数字にしようとしたが、ここで測り方を二度も間違えた。最初の方法では計算量を100倍にしても時間が変わらず、公式の性能計測機能を使っても同じだった。三度目でようやく実際の計算を捉えられた。その経緯も残す。

## ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

| ノード  | タイプ           | デフォルトから変更したパラメータ                        |
|------|---------------|-----------------------------------------|
| grid | grid          | cols = 20<br>rows = 20                  |
| wave | attribwrangle | snippet = @P.y = sin(@P.x * 2.0) * 0.5; |

## 実行対象を変えると、作られるアトリビュートの種類が変わる

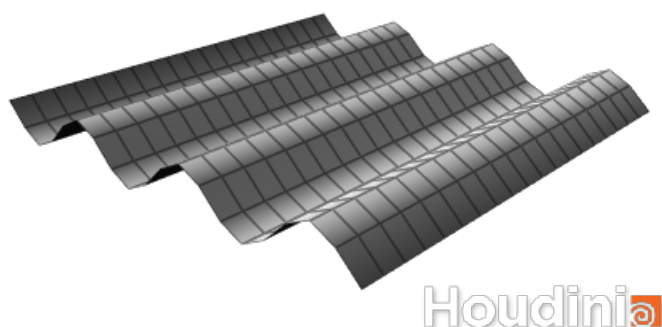
20×20 の grid（ポイント400 / プリミティブ361 / バーテックス1444）に対して、同じコード `@marker = 1.0;` を実行対象だけ変えて走らせた。

| 実行対象 (class)       | 作られたアトリビュートの種類 | 対象の要素数 |
|--------------------|----------------|--------|
| Detail (only once) | ディテール          | 1      |
| Primitives         | プリミティブ         | 361    |
| Points (既定)        | ポイント           | 400    |

| 実行対象 (class) | 作られたアトリビュートの種類 | 対象の要素数 |
|--------------|----------------|--------|
| Vertices     | バーテックス         | 1444   |
| Numbers      | (作られない)        | —      |

## 予測してから測る — $@P.y = \sin(@P.x * 2.0) * 0.5$

grid は 10×10 で原点中心なので x は -5 ~ +5。sin の値は -1 ~ +1 の範囲を取るので、y はちょうど -0.5 ~ +0.5 に収まるはず。



VEX 1行で作った波  $@P.y = \sin(@P.x * 2.0) * 0.5;$

|        | 値                     |
|--------|-----------------------|
| 予測した範囲 | -0.5 ~ +0.5           |
| 実測した範囲 | -0.499983 ~ +0.499983 |
| 差      | 0.000017              |

## 点数と処理時間（毎回新しいノードを作り、5回測った中央値）

1秒あたりの点数が点数とともに増えているのは、ノードを作る固定の費用が大きいため。点が増えるほどその費用が薄まる。

| 分割数  | 点数        | 中央値（ミリ秒） | 1秒あたりの点数 |
|------|-----------|----------|----------|
| 200  | 40,000    | 1.487    | 2,690万   |
| 400  | 160,000   | 2.005    | 7,979万   |
| 800  | 640,000   | 2.472    | 2億5,891万 |
| 1600 | 2,560,000 | 4.580    | 5億5,895万 |

## その計測は本物か — 同じ点数で計算量だけ増やす（64万点）

計算量を約100倍にしたときに時間が増えなければ、計測が実際の計算を捉えていないということ。結果の最大yが変わっていることも、確かに別の計算が走った証拠になる。

| 処理                 | 中央値（ミリ秒） | 軽いものとの比 | 結果の最大 y |
|--------------------|----------|---------|---------|
| 軽い (sin × cos を1回) | 2.48     | 1.0倍    | 0.49999 |

| 処理            | 中央値（ミリ秒） | 軽いものとの比 | 結果の最大 y |
|---------------|----------|---------|---------|
| 重い（100回の繰り返し） | 43.91    | 17.7倍   | 0.32169 |

## わかったこと

- ・ attribute wrangle の「実行対象（class）」は5種類ある。Detail (only once) / Primitives / Points / Vertices / Numbers で、既定は Points。
- ・ 実行対象を変えると、作られるアトリビュートの種類がそれに従って変わる。同じ @marker = 1.0; というコードでも、Points なら点に、Primitives なら面に、Vertices なら頂点に、Detail ならジオメトリ全体に1つだけ作られる。コードは同じでも結果の置き場所が変わるというのが、最初に押さえるべき点。
- ・ Numbers だけはアトリビュートが作られなかった。これはジオメトリの要素ではなく、指定した個数の番号に対して走るモードなので、値を書き込む先がないため。
- ・ 予測を立ててから測る形にした。sin の値域は -1 ~ +1 なので、0.5 倍すれば y は -0.5 ~ +0.5 に収まるはず。実測は -0.499983 ~ +0.499983 だった。
- ・ この 0.000017 のずれも説明できる。grid は20分割なので点の間隔は  $10/19 = 0.526316$ 。sin(2x) が最大になるのは  $x = \pi/4 \approx 0.785398$  のときだが、最も近い格子点は  $x = 0.789474$  で、わずかに外れている。この点での値を計算すると 0.4999834 になり、実測値と小数点以下6桁まで一致する。誤差ではなく、格子がピークをまたいでいるだけだった。
- ・ rand(@ptnum) は完全に再現する。同じコードを別のノードで2回走らせて、全400点が小数点以下9桁まで一致した。乱数といっても点番号から決まる固定の値なので、実行するたびに変わることはない。
- ・ その分布も妥当だった。0~2 の範囲に散らす設定で平均 1.03046（理論値 1.0）、範囲は 0.00103 ~ 1.99050。400点しかないので 3% ほどのずれは想定内。
- ・ 処理時間の計測は二度失敗した。一度目は cook(force=True) の前後を time.perf\_counter で挟む方法。これだと計算量を100倍にしても時間が 0.9 倍にしかならなかった。ジオメトリを要求した時点で初めて計算されるため、計測区間の外で処理が走っていた。
- ・ 二度目は Houdini の性能計測機能（hou.perfMon）を使ったが、軽い処理も重い処理も同じ 0.008000 秒を返した。同じノードを焼き直す限り、キャッシュが効いて実際には再計算されないらしい。
- ・ 三度目にようやく成功した。毎回新しいノードを作って初回のクックを測る。キャッシュが存在しないので必ず計算が走る。この方法だと計算量100倍の処理が 17.7倍 の時間になり、結果の最大yも 0.49999 から 0.32169 に変わった。時間と結果の両方が動いたので、今度は本物。
- ・ 得られた数字。64万点に sin × cos を1回かける処理が 2.47ミリ秒、256万点で 4.58ミリ秒。この2点から傾きを出すと、1点あたりの限界コストは約1.1ナノ秒（毎秒約9億点）、固定の費用が約1.8ミリ秒。
- ・ つまり「VEXが速い」効果は点数が多いときにしか出ない。4万点では1.49ミリ秒かかっており、そのほとんどが固定費。少ない点数なら、どんな書き方をしても体感は変わらない。
- ・ 17.7倍という比率が100倍にならないのは、繰り返しの中身が単純で、ノードを作る固定費も含まれているため。それでも計算量に応じて時間が動くことは確認できたので、計測としては成立している。

## 次にやること

- ・ for-each ループと VEX の使い分け。同じことをどちらでもできる場面で、書きやすさと速さを比べる
- ・ VEX からアトリビュートを読み書きする（point() / setpointattrib()）
- ・ 測り方の失敗が3回続いたので、計測の型を作っておく。「結果が変わること」と「時間が変わること」の両方を確認するのを標準にする

# for-each ループと VEX — 同じことをやって比べる

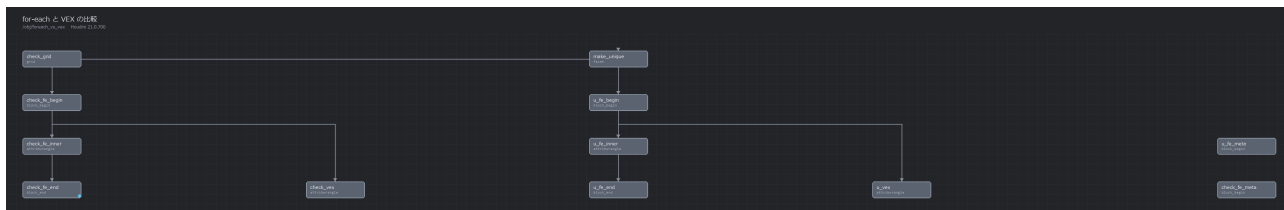
2026-09-13 | Houdini 21.0.700 | /obj/foreach\_vs\_vex | ノード 12 / 接続 9

## 何を作ったか

「まとめて処理できないときは for-each」とよく言われるが、VEX でも書ける場面でどれだけ差が出るのかは測らないと分からない。そこで、面ごとに違う高さへ動かすという同じ処理を2通りで作り、結果と速さを比べた。

ところが最初は結果が一致しなかった。その理由を追いかけた結果、この2つは「速い遅い」で選ぶものではなく、そもそも別のことをしているのだと分かった。

## ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

| ノード            | タイプ           | デフォルトから変更したパラメータ                                                                                                                           |
|----------------|---------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| check_grid     | grid          | cols = 8<br>rows = 8                                                                                                                       |
| check_fe_begin | block_begin   | blockpath = ../check_fe_end<br>method = 1                                                                                                  |
| check_fe_meta  | block_begin   | blockpath = ../check_fe_end<br>method = 2                                                                                                  |
| check_fe_inner | attribwrangle | snippet = @P.y += detail("op:../check_fe_meta", "iteration", 0) * 0.001;                                                                   |
| check_fe_end   | block_end     | blockpath = ../check_fe_begin<br>class = 0<br>itermethod = 1<br>method = 1<br>templatepath = ../check_fe_begin                             |
| check_vex      | attribwrangle | class = 1<br>snippet = int pts[] = primpoints(0, @primnum); foreach (int pt; pts) {<br>vector p = point(0, "P", pt); p.y += @primnum * ... |
| make_unique    | facet         | unique = 1                                                                                                                                 |
| u_fe_begin     | block_begin   | blockpath = ../u_fe_end<br>method = 1                                                                                                      |
| u_fe_meta      | block_begin   | blockpath = ../u_fe_end<br>method = 2                                                                                                      |
| u_fe_inner     | attribwrangle | snippet = @P.y += detail("op:../u_fe_meta", "iteration", 0) * 0.001;                                                                       |
| u_fe_end       | block_end     | blockpath = ../u_fe_begin<br>class = 0<br>itermethod = 1<br>method = 1<br>templatepath = ../u_fe_begin                                     |



| 分割数 | プリミティブ数 | for-each (ミリ秒) | VEX (ミリ秒) | 何倍    |
|-----|---------|----------------|-----------|-------|
| 32  | 961     | 53.55          | 1.414     | 37.9倍 |
| 64  | 3,969   | 191.87         | 2.233     | 85.9倍 |

## 結果が一致しなかった理由を追う

同じ処理のはずなのに、49面中48面で高さが違った。しかも差は全部 0.004 で一定。ここから3段階で原因を絞り込んだ。

| 調べたこと             | 結果                              | 分かったこと                                 |
|-------------------|---------------------------------|----------------------------------------|
| 出力の点数を比べる         | 元 64点 → for-each 196点 / VEX 64点 | for-each は面を1枚ずつ取り出すので、隣と共有していた点が複製される |
| VEX を3回走らせて結果を比べる | 3回とも完全に同じ                       | 複数の面が同じ点に書き込む競合ではあるが、順序は決まっていて再現する     |
| 点の共有をほどいてから比べる    | 完全に一致                           | 食い違いの原因は点の共有。ほどけば2つは同じ結果になる            |

## わかったこと

- ・ for-each は Block Begin (method = Extract Piece or Point) と Block End で処理を挟んで作る。Block End の Template Path が空だと「Missing template geometry」で止まる。何を何回繰り返すかはここから決まる。
- ・ 何回目かを知るには、メタデータ用の Block Begin (method = Fetch Metadata) をもう1つ置き、そのディテールアトリビュート iteration を読む。numiterations で総回数も取れる。
- ・ 取り出した中では @primnum が常に 0 になる。1個だけを切り出した状態で処理しているため。「何番目の面か」を使いたいならメタデータを読むしかない。
- ・ detail() のパスには op: が要る。detail("../メタデータノード", "iteration", 0) と書くとファイルパスとして扱われ、エラーも警告も出さずに 0 を返す。ループが動いていないように見えるので原因が分かりにくい。detail("op:../メタデータノード", ...) と書けば動く。実際にこれで1時間相当を溶かした。
- ・ for-each は面の数に比例して遅くなる。49面で11.79ミリ秒、3,969面で191.87ミリ秒。1面あたりおよそ0.05ミリ秒で、面の数がそのまま効く。
- ・ VEX はほとんど変わらない。49面で1.36ミリ秒、3,969面で2.23ミリ秒。実験009で測った「ノード1つあたり約1.8ミリ秒の固定費」がそのまま出ており、処理そのものの時間はまだ見えていない水準。
- ・ その結果、差は面の数とともに広がる。49面で 8.7倍、3,969面で 85.9倍。面が増えるほど for-each を選ぶ代償が大きくなる。
- ・ ただしこの2つは単純に置き換えられるものではない。for-each は面を1枚ずつ取り出すので、隣と共有していた点が複製される (64点 → 196点)。これは副作用ではなく、面ごとに独立して動かすために必要なこと。点を共有したままでは、隣り合う面が同じ点を別の場所へ動かそうとして取り合いになる。
- ・ その取り合いは競合ではあるが、結果は毎回同じだった (3回とも一致)。書き込む順序が決まっているため、実行のたびに変わることはない。最初は並列処理による非決定性を疑ったが、それは外れた。
- ・ 点の共有をほどいてから (facet の Unique Points で 64点 → 196点にしてから) 比べると、for-each と VEX の結果は完全に一致した。つまり食い違いの原因は処理内容ではなく、点が共有されているかどうか

かだけだった。

- ・実用的な結論。面ごとに独立した変形をしたいなら、まず点の共有をほどこ。ほどいてしまえば VEX で書ける場面が増え、数十倍速い。for-each が必要なのは、1回分の処理が VEX 1つでは書けないほど複雑なとき（中で複数のノードを通したいときなど）。

- ・測り方についても1つ。最初は面の中心を  $(x, y, z)$  でソートして比べようとしたが、比べたい量である  $y$  をソートの鍵に含めてしまったため、対応がずれて必ず食い違った。比べたい量は鍵にしない。水平位置  $(x, z)$  を鍵にして  $y$  を値にしたら正しく比較できた。

## 次にやること

- ・facet の Unique Points を挟むことで、for-each が必要だった処理をどこまで VEX に置き換えられるかを試す
- ・for-each の中で複数ノードを通す場合（VEX 1つでは書けない場合）の実例を作る
- ・polyextrude と boolean に進む

# crease で角を残す — どれだけの重みが必要なのか

2026-09-13 | Houdini 21.0.700 | /obj/crease\_test | ノード 15 / 接続 14

## 何を作ったか

実験002で「subdivide を重ねると立方体が球になる」と測った。3回分割すると一辺が 1.0 から 0.84918 まで縮む。crease（折り目）を付ければ角が保たれるはずだが、どれだけの重みが必要なのかは分からない。

理屈の上では「重み N なら N 回分の分割まで角が保たれる」はず。3回分割するなら重み3で完全に元の立方体のままになる、という予測を立てて確かめた。

結果は予測どおりだった。ただしこの実験では、2つの指標が違う答えを出すという厄介な状況にも当たった。

## ノードの接続

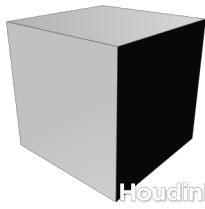


スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

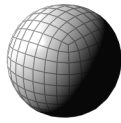
| ノード         | タイプ       | デフォルトから変更したパラメータ        |
|-------------|-----------|-------------------------|
| box1        | box       | (変更なし)                  |
| crease_0_0  | crease    | crease = 0.0<br>op = 1  |
| sub_0_0     | subdivide | iterations = 3          |
| crease_0_5  | crease    | crease = 0.5<br>op = 1  |
| sub_0_5     | subdivide | iterations = 3          |
| crease_1_0  | crease    | op = 1                  |
| sub_1_0     | subdivide | iterations = 3          |
| crease_2_0  | crease    | crease = 2.0<br>op = 1  |
| sub_2_0     | subdivide | iterations = 3          |
| crease_3_0  | crease    | crease = 3.0<br>op = 1  |
| sub_3_0     | subdivide | iterations = 3          |
| crease_4_0  | crease    | crease = 4.0<br>op = 1  |
| sub_4_0     | subdivide | iterations = 3          |
| crease_10_0 | crease    | crease = 10.0<br>op = 1 |
| sub_10_0    | subdivide | iterations = 3          |

## 重みを変えて3回分割する

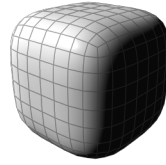
すべて同じカメラ。点数・面数はどの重みでも 386点 / 384面で変わらない。crease は形の計算方法を変えるだけで、ジオメトリを増やさない。



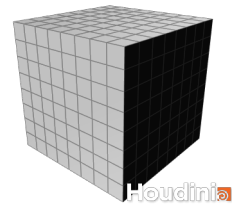
分割前の box 一辺  
1.00000 / 折れ角 90.00度



重み 0.0 (crease なし)  
一辺 0.84918 / 折れ角  
13.93度



重み 1.0 一辺 1.00000 /  
折れ角 28.07度

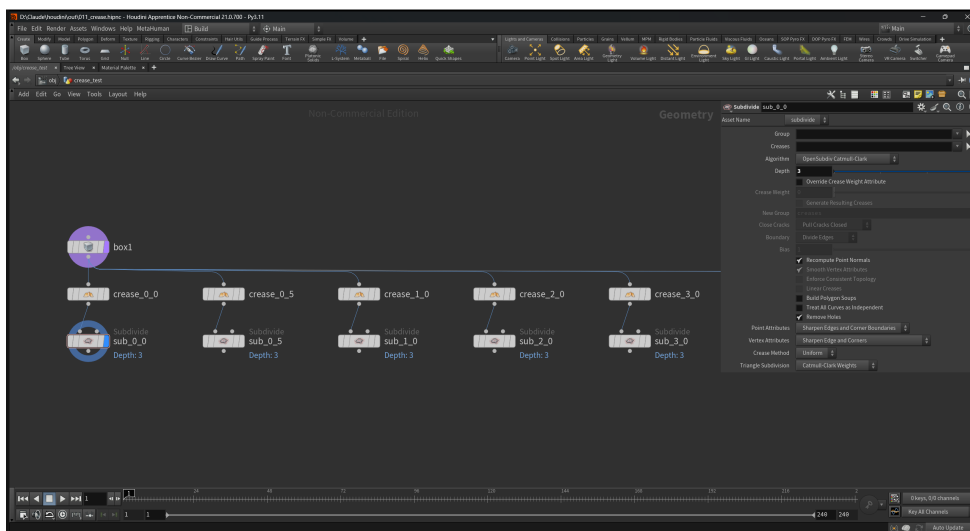


重み 3.0 一辺 1.00000 /  
折れ角 90.00度

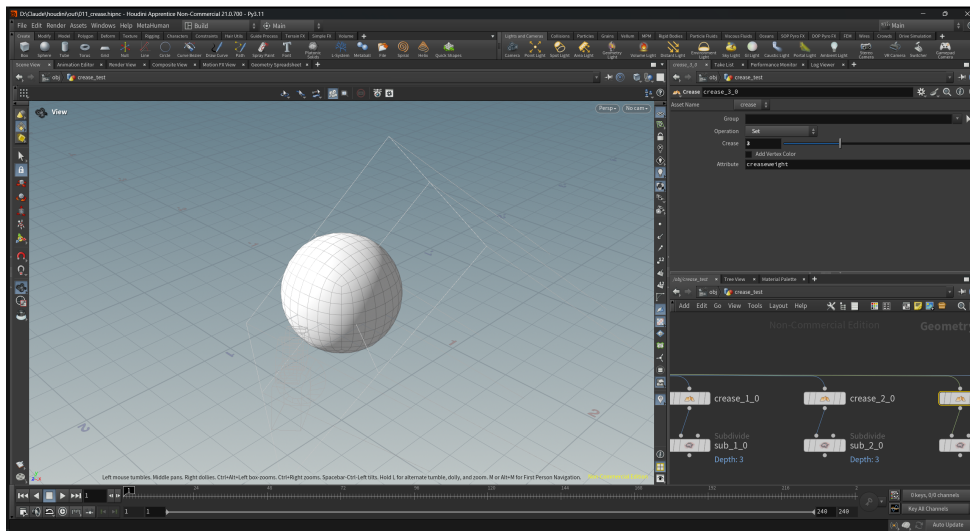
| crease の重み    | 点数  | 面数  | 一辺の長さ   | 最大の折れ角 |
|---------------|-----|-----|---------|--------|
| 0.0 (なし)      | 386 | 384 | 0.84918 | 13.93度 |
| 0.5           | 386 | 384 | 0.92459 | 21.45度 |
| 1.0           | 386 | 384 | 1.00000 | 28.07度 |
| 2.0           | 386 | 384 | 1.00000 | 53.13度 |
| 3.0           | 386 | 384 | 1.00000 | 90.00度 |
| 4.0           | 386 | 384 | 1.00000 | 90.00度 |
| 10.0          | 386 | 384 | 1.00000 | 90.00度 |
| (参考) 分割前の box | 8   | 6   | 1.00000 | 90.00度 |

## 実際の Houdini の画面

crease SOP で折り目の重みを付け、その下流の subdivide がそれを読む。crease SOP の Crease Attribute の既定は creaseweight で、subdivide はこの名前の属性を自動で見ると。



box から crease と subdivide の組を7通り並べた



crease SOP のパラメータ。Operation を Set にして Crease に重みを入れる

## わかったこと

- ・予測は当たった。3回分割する場合、重み 3.0 で折れ角が 90.00度 になり、分割前の box とまったく同じ角の鋭さになる。「重み N なら N 回分の分割まで角が保たれる」という理解でよい。
- ・重みを 3 より上げて何も変わらない (4.0 も 10.0 も 90.00度 のまま)。分割回数を超える重みは無駄。逆に言えば、分割回数を後から増やすつもりなら重みも合わせて上げる必要がある。
- ・点数・面数はどの重みでも 386点 / 384面で変わらなかった。crease は形の計算方法を変えるだけで、ジオメトリを増やさない。角を残すためにポリゴンを足す必要はない。
- ・2つの指標が違う答えを出した。一辺の長さは重み 1.0 で 1.00000 に達し、そこから変わらない。この指標だけを見ていたら「重み1で十分」と判断してしまう。しかし折れ角は 28.07度 で、まだ元の 90度 には程遠い。画を見ると角がはっきり丸い。
- ・つまり一辺の長さは「どこまで縮んだか」しか測っておらず、「角が鋭いかどうか」は測れていない。重み1で外形は元に戻るが、角は丸いまま。何を知りたいかによって、見るべき指標が変わる。
- ・折れ角の指標は、辺を共有する2つの面の法線のなす角の最大値として作った。角が残っていれば大きく、丸まれば小さくなる。分割前の box で 90.00度 と出るので、基準としても正しく働いている。
- ・重み 0.5 のとき一辺は 0.92459 だった。これは crease なし (0.84918) と重み1.0 (1.00000) のちょうど中間で、小数点以下5桁まで一致する。小数の重みは2つの結果を線形に混ぜている。
- ・ただし折れ角のほうは線形ではない。0.0 / 0.5 / 1.0 で 13.93 / 21.45 / 28.07度 となり、中間値の 21.00度 とはわずかにずれる。位置は線形に混ざるが、角度はそうならない。
- ・crease SOP の Operation の既定は「Add To Existing Value」で、既にある値に足し込む。重みを決め打ちしたいなら「Set」にする。既定のまま何度も通すと重みが積み上がる。
- ・crease SOP が書き込む属性名の既定は creaseweight で、subdivide はこの名前を自動で見る。名前を変えたい場合は subdivide 側の Creases にも同じ名前を指定する必要がある。

## 次にやること

- ・一部の辺だけに crease を付けて、角を残す場所と丸める場所を作り分ける
- ・polyextrude で作った形に crease を付け、建物のような硬い形を作る

- ・折れ角の指標を、面ごとではなく辺ごとに出して分布を見る（どの辺が丸まっているかを特定する）